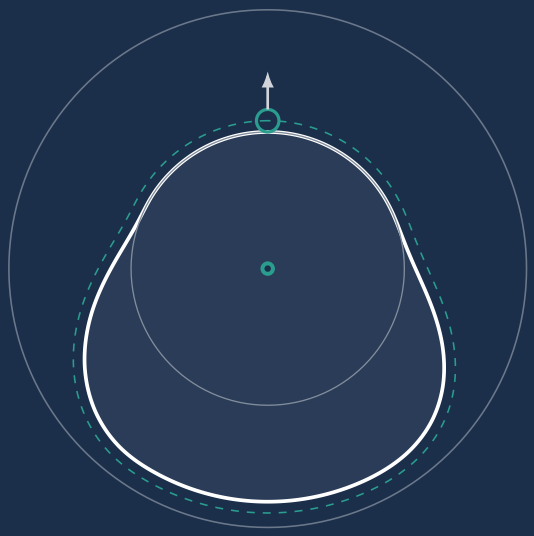
ELECTRONIC CAMMING ON THE TEKNIC CLEARCORE

Electronic CAM Workbench

Cam Law Mathematics and Theory of Operation



From the motion laws to synchronized multi-axis motion with Teknic ClearPath servos

Software version 0.11.1 | ECW-TOP-100, Revision A

AUTHOR AND DEVELOPER

Adam G. Sweeney

agsweeney@gmail.com

Written in ASD-STE100 Simplified Technical English.
Edited to *The Chicago Manual of Style*, 18th edition.

November 2026

MIT License

Electronic CAM Workbench: Cam Law Mathematics and ClearCore Theory of Operation
Document ECW-TOP-100, Revision A, November 2026
For Electronic CAM Workbench and ECam firmware version 0.11.1

Copyright © 2026 Adam G. Sweeney. All rights reserved.

The Electronic CAM Workbench and its ECam firmware are released under the MIT License. The full license text is in the file `LICENSE` at the root of the source repository. The software is supplied "as is," without warranty of any kind.

Teknic®, ClearCore™, and ClearPath® are trademarks of Teknic, Inc. EtherNet/IP™ and CIP™ are trademarks of ODVA, Inc. Micro800 and Micro850 are trademarks of Rockwell Automation, Inc. ASD-STE100® is a trademark of ASD (AeroSpace, Security and Defence Industries Association of Europe). All other product names are trademarks of their respective owners. Their use in this document identifies products and does not imply endorsement. Teknic, Inc. does not publish or support this document.

Status of this document. This document gives the design of the software. It is not a record of tests. The results of the bench tests are in `docs/BENCH_STATUS.md` and in the test reports in `docs/`.

Revision	Change
A, November 2026	First issue, for software version 0.11.1.

Contents

Figures vii

Tables vii

Preface xi

Notation xv

1 Introduction to Electronic Cams 1
- 1.1 The Mechanical Cam . . . 1
- 1.2 The Electronic Cam . . . 1
- 1.3 The Electronic CAM Workbench . . . 1
- 1.4 Design Principles . . . 2
- 1.5 Functions . . . 3
- 1.6 The Source of the Equations . . . 3
- 1.7 Limitations . . . 3

I Cam Mathematics 5

2 Masters, Followers and Segments 7
- 2.1 The Master and the Follower . . . 7
- 2.2 Segments and Knots . . . 7
- 2.3 The Normalized Law . . . 8
- 2.4 From the Unit Square to the Segment . . . 8
- 2.5 From the Master to Time . . . 8

3 The Motion Laws 11
- 3.1 Properties of a Law . . . 11
- 3.2 Dwell . . . 11
- 3.3 Linear . . . 11
- 3.4 Simple Harmonic . . . 12
- 3.5 Cycloidal . . . 12
- 3.6 Piecewise Sinusoidal Laws . . . 13
- 3.7 Polynomial Laws . . . 14
- 3.8 Blend and Cubic Spline . . . 14
- 3.9 Comparison of the Laws . . . 15
- 3.10 Selection of a Law . . . 15

4 Building a Profile 17
- 4.1 The Data of an Axis . . . 17
- 4.2 Ties Between Knots . . . 17
- 4.3 Repeating and One-Shot Axes . . . 17

4.4 Finding the Position . . . 18
4.5 Rules That Keep a Profile Valid . . . 18
4.6 Continuity at the Knots . . . 18
4.7 The Electronic Gear . . . 19

5 Blend and the Cubic Spline 21
5.1 Blend . . . 21
5.2 The Cubic Spline . . . 22

6 Analysis and Speed Limits 25
6.1 Sampling . . . 25
6.2 The Peak Values . . . 25
6.3 From Axis Units to the Motor . . . 25
6.4 An Example: The Carton Pusher . . . 26
6.5 The Fastest Safe Speed . . . 26
6.6 The Checks . . . 27

7 Plate Cam Geometry 29
7.1 The Frame . . . 29
7.2 The Pitch Curve . . . 29
7.3 The Cam Surface . . . 29
7.4 The Pressure Angle . . . 30
7.5 Curvature and Undercut . . . 31

8 Cut-Length Mode 33
8.1 The Terms . . . 33
8.2 The Wait at the Start of the Cycle . . . 33
8.3 Stretching the Cycle . . . 33
8.4 An Example: The Variable-Length Cutoff . . . 34
8.5 Changing the Length . . . 34
8.6 The Gap to the Cut Line . . . 34

II Theory of Operation on the ClearCore 37

9 The System 39
9.1 The Parts . . . 39
9.2 One Code for the PC and the ClearCore . . . 39
9.3 The EnhancedClearCoreLibrary . . . 40

10 The Sample Loop 41
10.1 The Rate . . . 41
10.2 The Order in a Sample . . . 41
10.3 The Main Loop . . . 41
10.4 Fixed-Point Numbers . . . 42

11 The Masters **43**
11.1 Master Units . . . 43
11.2 The Encoder Master . . . 43
11.3 The Driven Crank . . . 43
11.4 The Timeline . . . 44
11.5 Simulation . . . 45

12 Followers and Step Generation **47**
12.1 The Position Tables . . . 47
12.2 Interpolation . . . 47
12.3 Steps Are Differences . . . 47
12.4 The Electronic Gear at Run Time . . . 48
12.5 Step Output in the Library . . . 48

13 Running a Machine **49**
13.1 The States . . . 49
13.2 Start . . . 49
13.3 The Cam Position . . . 50
13.4 Stop and Abort . . . 50
13.5 Enable and Disable . . . 50
13.6 Jogs . . . 50
13.7 Homing . . . 50
13.8 Protection . . . 51

14 Profile Storage and Integrity **53**
14.1 The Text Format . . . 53
14.2 The Hash . . . 53
14.3 The Commit . . . 53
14.4 The Flash Store . . . 53

15 Control from a PLC **55**
15.1 The Assemblies . . . 55
15.2 Who Has Control . . . 55
15.3 Loss of the Connection . . . 56
15.4 A Typical Sequence . . . 56

A Coefficients of the Laws **57**

Glossary **59**

Bibliography **63**

This page is intentionally left blank.

List of Figures

1.1 An electronic cam. Each axis follows its own profile of the same master position. 1

2.1 The displacement diagram of the carton pusher. The dots are the knots: the ends of the segments. . . . 7

3.1 Simple harmonic. The acceleration does not go to zero at the ends. . . . 12
3.2 Cycloidal. The acceleration is zero at both ends. . . . 12
3.3 Modified trapezoid. The flat tops give the low peak acceleration. . . . 13
3.4 Modified sine. The low velocity peak gives a low peak motor speed. . . . 14
3.5 Polynomial 3-4-5. . . . 15
3.6 Polynomial 4-5-6-7. The jerk is zero at both ends. . . . 15
3.7 The acceleration of the laws for the same rise and span. . . . 16

4.1 The velocity and acceleration of the carton pusher, per degree of the crank. The marks show the knots. . . . 18

5.1 The carriage of a flying saw. The shaded band is the linear segment: the sync. The Blends meet it at its velocity. . . . 22
5.2 A periodic cubic spline through its knots, and its acceleration: continuous and linear between the knots. . . . 24

6.1 The speed limit of the carton pusher at each crank angle. In the dwells, the axis does not move and has no limit. . . . 27

7.1 The carton pusher as a plate cam. Solid: the cam surface. Dashed: the pitch curve. Thin: the base circle. The roller is shown at $\theta = 0$. . . . 30
7.2 The pressure angle of the carton pusher. The shaded bands are above the 30° limit. 30

8.1 The carriage and the saw plunge for two piece lengths. The longer piece adds line travel to the wait only. . . . 34

9.1 The parts of the system. . . . 39

10.1 One sample. The callback calculates the steps. The library loads them, and they go out in the next period. . . . 41

11.1 The speed of a driven crank. The acceleration term $y'\alpha$ of equation (2.8) applies on the ramps. . . . 44
11.2 The three axes of a pick-and-place cycle on one timeline. . . . 45

12.1 Steps per sample of the carton pusher at 32 rpm (dotted: the exact rate). The limit of the ClearCore is 100 steps per sample. . . . 48

13.1 The states of the crank and the timeline. . . . 49

This page is intentionally left blank.

List of Tables

P.1 Where to start xii
P.2 Typographic conventions xiii

1.1 Mechanical and electronic cams 2
1.2 Functions 3

3.1 The peak coefficients and end conditions of the laws. 16

6.1 Counts per unit for each drive type 25
6.2 The carton pusher at 32 rpm 26
6.3 The checks of the analysis 27

10.1 The fixed-point values in the sample loop. 42

13.1 The protection functions 51

15.1 The assemblies of the ClearCore 55
15.2 The control bits of the command assembly 55

A.1 The peak coefficients of the laws 57

This page is intentionally left blank.

Preface

Purpose of This Document

This document gives the mathematics of the Electronic CAM Workbench and its theory of operation on the Teknic ClearCore. It tells you how the Workbench calculates a cam profile, how it checks the profile against the machine, and how the ClearCore runs the profile.

The Workbench replaces the mechanical cams of a machine with *electronic cams*. You draw the motion of each axis against one *master*: a shaft of the machine, a motor that the ClearCore turns, or a clock. The ECam firmware on the ClearCore then moves up to four Teknic ClearPath servos in exact step with that master. It runs without the PC, and a PLC can control it through EtherNet/IP.

The document starts with the mathematics, which does not depend on a controller. It then shows how the ClearCore applies that mathematics, sample by sample. Each equation is the equation that the code calculates, and each chart comes from the code.

Audience

This document is for these engineers:

- Machine designers who select the motion laws and the timing of a machine.
- Commissioning and service engineers who must know why the Workbench accepts or refuses a profile, and what the ClearCore does at each step.
- Software engineers who change the Workbench, the ECam firmware, or the Enhanced-ClearCoreLibrary.

You must know differential calculus, the terms of motion control (position, velocity, acceleration, and jerk), and the basics of step-and-direction servo drives. Part II also uses the terms of fixed-point arithmetic. The glossary at the end of the document defines the terms that the document uses.

How to Use This Document

Table P.1 shows where to start for each task.

Conventions

Safety Notices

△ WARNING

A warning tells you about a condition that can cause injury or death. It also tells you about a condition that can make machinery move when you do not expect it.

Table P.1. Where to start

Task	Read
Learn what an electronic cam is and what the Workbench does	Chapter 1
Learn the terms: master, follower, segment, and law	Chapter 2
Select a motion law for a segment	Chapter 3
Join segments, use Blend, or fit a spline through points	Chapters 4 and 5
Find the fastest safe speed of a machine	Chapter 6
Check a plate cam for its pressure angle and undercut	Chapter 7
Set up a flying saw with a variable piece length	Chapter 8
Learn how the ClearCore keeps all axes in step	Chapters 9 to 12
Learn what start, stop, jog, homing, and the limits do	Chapter 13
Learn how a profile is stored and checked	Chapter 14
Control the machine from a PLC	Chapter 15
Look up a coefficient of a law, a symbol, or a term	Appendix A, the notation, and the glossary

CAUTION

A caution tells you about a condition that can cause damage to equipment or incorrect motion.

NOTE: A note gives information that helps you. It does not tell you about a hazard.

A box with a teal frame gives the key result of a section.

Typography

Table P.2. Typographic conventions

Example	Meaning
`Evaluate`, `diag`	A function in the code, or a command of the ClearCore console.
`core/CamProfile.cpp`	A file or a directory, relative to the root of the source repository.
Axis	A menu, a dialog, or a field in the Workbench.
knot	A term at its first use. The glossary defines it.
y', $\dot{y}$	A derivative with respect to the master, and with respect to time.
equation (2.3)	Equation 3 of chapter 2. Figures and tables have the same numbers.

Angles of the master are in degrees. Times are in seconds. A speed of the crank is in revolutions per minute (rpm). An axis position is in the unit of that axis, for example millimetres or degrees. A motor position is in *counts*: steps of the motor. The notation on page xv lists the symbols.

Related Documents

- The user guide of the Workbench. Open it with the Help button or the F1 key. It tells you how to use the application.
- `README.md`: an overview of the Workbench, its masters, outputs, and example projects.
- `docs/MATH.md`: a short form of the mathematics in Part I.
- `Firmware/PROTOCOL.md`: the console commands of the ECam firmware and the EtherNet/IP assemblies.
- `MODIFICATIONS.md` in the EnhancedClearCoreLibrary: the changes to the Teknic library that camming needs.
- *QuickEIP Programming and Implementation Manual*, QE-PIM-200: the EtherNet/IP stack of the firmware.

If this document and the code are different, the code is correct. Use the code, and report the difference.

This page is intentionally left blank.

Notation

The symbols below have the same meaning in the full document. A prime ($'$) is a derivative with respect to the master position. A dot ($\dot{}$) is a derivative with respect to time.

Symbol	Meaning	Unit
θ	Master position on a crank: the crank angle	deg
t	Master position on a timeline: the time from the start of the cycle	s
x	Master position, θ or t, when the text applies to both	deg or s
X	Span of the master cycle: 360° on a crank, the cycle duration on a timeline	deg or s
u	Normalized master position in a segment, $0 \le u \le 1$	–
$s(u)$	Normalized displacement of a motion law, $0 \le s \le 1$ on a rise	–
s', s'', s'''	Derivatives of s with respect to u	–
C_v, C_a, C_j	Peak coefficients: the largest $\lvert s'\rvert$, $\lvert s''\rvert$, $\lvert s'''\rvert$ of a law	–
$y(x)$	Follower position (the axis position)	mm, deg, ...
y', y'', y'''	Follower velocity, acceleration and jerk with respect to the master	unit/deg^n or unit/s^n
x_0, x_1	Start and end of a segment on the master	deg or s
y_0, y_1	Position of the follower at x_0 and x_1	axis unit
β	Span of a segment, $\beta = x_1 - x_0$	deg or s
h	Rise of a segment, $h = y_1 - y_0$	axis unit
ω	Master speed: $\omega = 6N$ for a crank speed N in rpm	deg/s
α	Master acceleration, for a crank that the ClearCore ramps	deg/s^2
k	Speed override of a timeline (1 is the speed as drawn)	–
c	Counts per unit of an axis: motor steps for one axis unit	count/unit
f_s	Sample rate of the motion loop	Hz
R_b, R_r	Base radius and roller radius of a plate cam	axis unit
ϕ	Pressure angle of a plate cam	deg
ρ	Radius of curvature	axis unit

This page is intentionally left blank.

CHAPTER 1

Introduction to Electronic Cams

1.1 The Mechanical Cam

A machine that makes the same product many times does the same motions in each cycle. A packaging machine pushes a carton, folds a flap, and seals it. A flying saw follows the material, cuts it, and returns. In each cycle, each motion must occur at the correct time, relative to all the other motions.

For a long time, mechanical cams did this work. One main shaft turns once for each cycle. A plate cam on the shaft has a shaped edge, and a roller follower rides on it. The shape of the edge sets the position of the follower at each angle of the shaft. All the cams are on one shaft, so all the followers stay in step: if the shaft turns slower, every motion becomes slower by the same amount.

A mechanical cam is reliable, but it is difficult to change. A new motion needs a new cam. A new product size can need a full set of new cams, and time to install them.

1.2 The Electronic Cam

An electronic cam keeps the principle and replaces the hardware. The shape of the cam becomes a function in software: the position of each axis as a function of the master position. A servo motor moves each axis. A controller reads the master position many times each second and sends each motor to the position that its function gives.

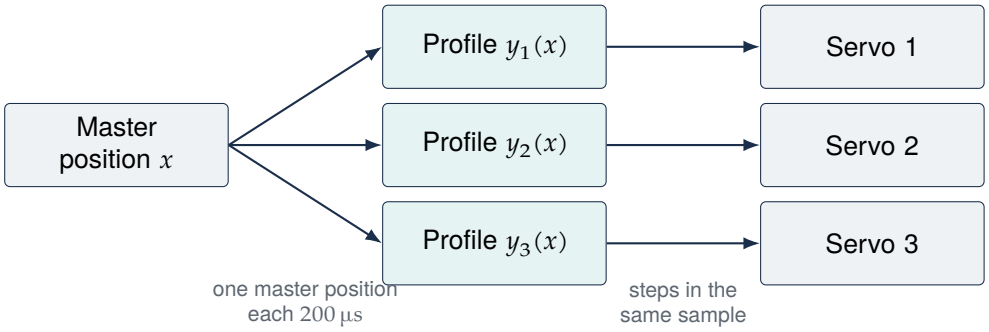


Figure 1.1. An electronic cam. Each axis follows its own profile of the same master position.

Table 1.1 compares the two.

The electronic cam has one new risk. The mechanical cam always holds its follower. The electronic cam depends on the controller, the drives, and the software. Thus this document gives the design of each part in detail.

1.3 The Electronic CAM Workbench

The Workbench is two programs that use one code:

- **The Workbench application** on a PC. You draw the profile of each axis, select its motion

Table 1.1. Mechanical and electronic cams

	Mechanical cam	Electronic cam
Shape of the motion	Cut into the plate	A list of segments in software
Change of the motion	Make and install a new cam	Load a new profile
Synchronization	The shaft connects all cams	One master position for all axes
Master	The main shaft	A shaft encoder, a driven crank motor, or a clock
Speed limit	Wear, pressure angle, and follower bounce	The speed, acceleration, and step rate of each servo
Stroke	Limited by the size of the cam	Limited by the travel of the axis

laws, and set the mechanics of the axis. The Workbench checks the profile and shows its velocity, acceleration, jerk, and speed limits. It also simulates the machine, exports plate cams and paths as DXF files, and makes reports.

- **The ECam firmware** on a Teknic ClearCore. It keeps the profile in flash memory and runs it. Each 200 µs it reads the master and sends the steps of each axis to its Teknic ClearPath servo.

The firmware uses the EnhancedClearCoreLibrary, our extension of the Teknic ClearCore library. The extension lets one function set the steps of every motor in each sample. The stock library cannot keep several motors in step with an external master, because each motor plans its own move.

1.4 Design Principles

Four rules control the design.

1. **One code for the PC and the ClearCore.** The laws, the profile, the analysis, and the run-time engine are in `core/`. The Workbench and the firmware compile the same files. Thus the curve on the screen is the curve that the machine runs, and the simulation runs the engine of the firmware.
2. **Whole numbers at run time.** The ClearCore builds a table of whole motor counts for each axis when it receives a profile. In the motion loop it uses only whole numbers and fixed-point fractions. The steps of a sample are the change of a whole-number target, so an axis cannot gain or lose a count through rounding.
3. **One master position for all axes.** In each sample the firmware calculates one master position, the target of every axis, and the steps of every motor. All the steps go out in the same period. No axis waits for another.
4. **The ClearCore runs alone.** The profile is in flash memory, and the ClearCore loads it at power-up. A PLC can start, stop, and adjust the machine through EtherNet/IP. The PC is necessary only to design and to load a profile.

1.5 Functions

Table 1.2 lists what the Workbench and the firmware do.

Table 1.2. Functions

Function	Details
Masters	An encoder on the machine, a crank motor that the ClearCore turns with a set speed and ramp, or a timeline (a clock) with a trigger. A project can use a crank and a timeline together.
Axes	Up to four ClearPath servos on the motor connectors M-0 to M-3. Each axis has its own unit, mechanics (direct, ball screw, or belt), and limits of velocity and acceleration.
Motion laws	Dwell, linear, simple harmonic, cycloidal, modified trapezoid, modified sine, polynomial 3-4-5, polynomial 4-5-6-7, Blend, and cubic spline. Up to 64 segments for each axis.
Electronic gear	An axis that follows the crank at a fixed ratio, exact in whole numbers.
Cut-length mode	A flying saw: the cycle stretches to the length of the piece, which a PLC can change during production.
Outputs and waits	Pulses on the outputs IO-0 to IO-5 at set angles or times. A timeline can wait for an input, a PLC bit, or a CCIO-8 input.
Machine functions	Start, cycle stop, jogs, homing to a switch, limit switches, an emergency stop input, and checks of the motor HLFB signals.
Analysis	Peak values, motor speed and step rate, the fastest safe speed, steps in velocity or acceleration at the knots, and the pressure angle and undercut of a plate cam.
PLC control	An EtherNet/IP adapter with one Class 1 connection: a 64-byte command and a 160-byte status.
Exports	Plate cams and axis paths as DXF files (R2000), in millimetres or inches, and project reports.

1.6 The Source of the Equations

The equations in this document are the equations in the code, and the document gives the name of the function where this helps you find it. A tool, `tools/docs_data`, calculates the data for each chart with the core code, from the example projects in `samples/`. Thus each curve in this document is a curve that the Workbench and the ClearCore calculate. The numbers in the examples come from the same projects.

1.7 Limitations

Compare these limits with the needs of your machine before you design with the Workbench.

- The ClearCore has four motor connectors. A driven crank motor uses one of them.
- An axis has at most 64 segments. A cubic spline thus has at most 65 knots.

- A motor receives at most 100 steps in each sample: 500,000 steps per second. Jogs and homing use at most 53,333 counts per second.
- The ClearCore does not read the position of a ClearPath servo. It sees the HLFB signal only, which shows a fault or a servo that is not ready. Each servo closes its own position loop.
- The ClearCore cannot ramp an encoder master, because the machine turns it. When you stop the cycle, the axes on an encoder master stop at once.
- The analysis checks the profile against the limits that you enter for each axis. It does not know the inertia, the friction, or the compliance of the mechanics.

△ WARNING

The ECam firmware is not a safety function. Its emergency stop input disables the motors, but the firmware reads it in the main loop, and a fault of the software can prevent the stop. Use a certified safety circuit that removes the power to the motors to stop the machine in an emergency.

PART I

Cam Mathematics

This page is intentionally left blank.

CHAPTER 2

Masters, Followers and Segments

An electronic cam replaces a mechanical cam with software. A *master* sets the progress of the machine cycle. A *follower* is an axis. The position of each follower is a function of the master position. This chapter gives the terms and the one equation that all motion laws use.

2.1 The Master and the Follower

The master is one of two kinds:

- **A crank.** The master position is an angle θ, from 0° to 360° for each machine cycle. An encoder on the shaft of the machine gives the angle. Or the ClearCore turns a crank motor and knows the angle. The cycle repeats each revolution.
- **A timeline.** The master position is a time t, from 0 s to the cycle duration X. A clock in the ClearCore gives the time. A trigger starts the cycle. The cycle can run once or repeat.

The text uses x for the master position when it applies to both kinds, and X for the span of the cycle: $X = 360°$ on a crank.

A follower position is a function

$$y = y(x), \qquad 0 \le x \le X. \tag{2.1}$$

The function is the *profile* of the axis. Its graph is the *displacement diagram*. Figure 2.1 shows the profile of the carton pusher in the example `packaging_machine`. The pusher dwells, rises 120 mm, dwells, and returns.

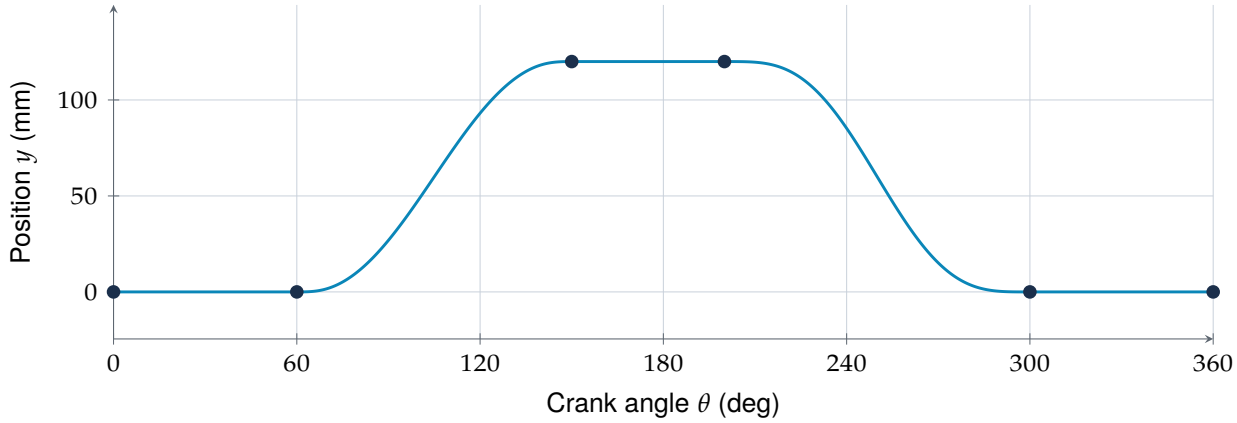


Figure 2.1. The displacement diagram of the carton pusher. The dots are the knots: the ends of the segments.

2.2 Segments and Knots

A profile is a sequence of *segments*. Each segment covers a part of the master cycle, from x_0 to x_1, and moves the follower from y_0 to y_1. The ends of the segments are the *knots*. Knot 0 is at $x = 0$.

Each segment starts where the previous segment ends, so the positions make a chain.

Each segment has a *motion law*. The law gives the shape of the motion between the two knots. Chapter 3 gives each law.

A segment has two dimensions:

$$\beta = x_1 - x_0 \qquad \text{the span of the segment on the master,} \tag{2.2}$$

$$h = y_1 - y_0 \qquad \text{the rise of the segment (negative for a return).} \tag{2.3}$$

2.3 The Normalized Law

A motion law is defined once, on a unit square. The input is the normalized master position

$$u = \frac{x - x_0}{\beta}, \qquad 0 \le u \le 1. \tag{2.4}$$

The output is the normalized displacement $s(u)$. A rise law has $s(0) = 0$ and $s(1) = 1$. The law also gives the derivatives $s'(u)$, $s''(u)$ and $s'''(u)$ with respect to u.

The *dwell* is the exception: $s(u) = 0$ for all u, and the profile ties its two knots to the same position.

2.4 From the Unit Square to the Segment

The follower position in a segment is

$$y(x) = y_0 + h\,s(u). \tag{2.5}$$

The chain rule gives the derivatives with respect to the master. Each derivative divides by one more power of β:

$$y' = \frac{h}{\beta}\,s'(u), \qquad y'' = \frac{h}{\beta^2}\,s''(u), \qquad y''' = \frac{h}{\beta^3}\,s'''(u). \tag{2.6}$$

On a crank, the unit of y' is axis units per degree. On a timeline, it is axis units per second of the diagram.

> Equations (2.5) and (2.6) apply to every law except Blend and the cubic spline. Blend uses its own basis with the same $1/\beta^n$ scale (section 5.1). The spline is calculated directly in master units (section 5.2).

2.5 From the Master to Time

The motors move in time, not in degrees. The master speed ω connects the two:

- On a crank that turns at N revolutions per minute, $\omega = 6N$ degrees per second.
- On a timeline, $\omega = k$, the speed override. With $k = 1$ the timeline runs as drawn.

At a constant master speed, the derivatives with respect to time are

$$\dot{y} = y'\,\omega, \qquad \ddot{y} = y''\,\omega^2, \qquad \dddot{y} = y'''\,\omega^3. \tag{2.7}$$

The velocity increases with the speed. The acceleration increases with the square of the speed. Thus a machine that runs twice as fast needs four times the acceleration from each axis.

When the ClearCore ramps a driven crank at α degrees per second squared, the acceleration has a second term:

$$\ddot{y} = y'' \, \omega^2 + y' \, \alpha. \tag{2.8}$$

Chapter 6 uses equations (2.7) and (2.8) to find the fastest safe speed of the machine.

> **NOTE:** An encoder master and a timeline have $\alpha = 0$ in the analysis. The machine sets the speed of an encoder master. The Workbench cannot ramp it.

This page is intentionally left blank.

CHAPTER 3

The Motion Laws

A motion law gives the shape of a segment on the unit square of chapter 2. The Workbench has ten laws. This chapter gives the equations of each law, as the code calculates them, and the properties that decide where you can use the law.

3.1 Properties of a Law

Three properties decide the quality of a law:

- **The peak coefficients.** C_v, C_a and C_j are the largest values of $|s'|$, $|s''|$ and $|s'''|$ on $0 \le u \le 1$. With equation (2.6), the peak velocity of a segment is $C_v\, h/\beta$ and its peak acceleration is $C_a\, h/\beta^2$.
- **The end conditions.** A law that has $s'(0) = s'(1) = 0$ starts and ends at rest. It can join a dwell without a step in the velocity. A law that also has $s''(0) = s''(1) = 0$ joins a dwell without a step in the acceleration.
- **The continuity inside the law.** A step in the acceleration gives an infinite jerk: a shock to the machine. A step in the velocity gives an infinite acceleration: a hard knock.

Each chart in this chapter shows s, and the derivatives divided by their peak coefficients, so all four curves fit between -1 and 1.

3.2 Dwell

A dwell holds the follower at one position:

$$s(u) = s'(u) = s''(u) = s'''(u) = 0. \tag{3.1}$$

The profile ties the two knots of a dwell to the same position. If you move one knot, the other knot moves with it.

3.3 Linear

The linear law has a constant velocity:

$$s = u, \qquad s' = 1, \qquad s'' = s''' = 0. \tag{3.2}$$

The velocity has a step at each end, unless the neighbours move at the same velocity. Use the linear law for a constant-velocity part of a motion between two Blend segments, for example the sync of a flying saw. Do not use it next to a dwell.

CAUTION

The values of the linear law do not show the steps at its ends: s'' is zero everywhere. The analysis (chapter 6) finds the steps at the knots.

3.4 Simple Harmonic

The simple harmonic law is half a cosine wave:

$$s = \frac{1}{2}\left(1 - \cos \pi u\right), \quad s' = \frac{\pi}{2} \sin \pi u, \quad s'' = \frac{\pi^2}{2} \cos \pi u, \quad s''' = -\frac{\pi^3}{2} \sin \pi u. \tag{3.3}$$

It starts and ends at rest. But its acceleration is $\pi^2/2$ at $u = 0$ and $-\pi^2/2$ at $u = 1$. Thus the acceleration has a step where the law meets a dwell. The code gives zero jerk at the ends. The real jerk there is an impulse.

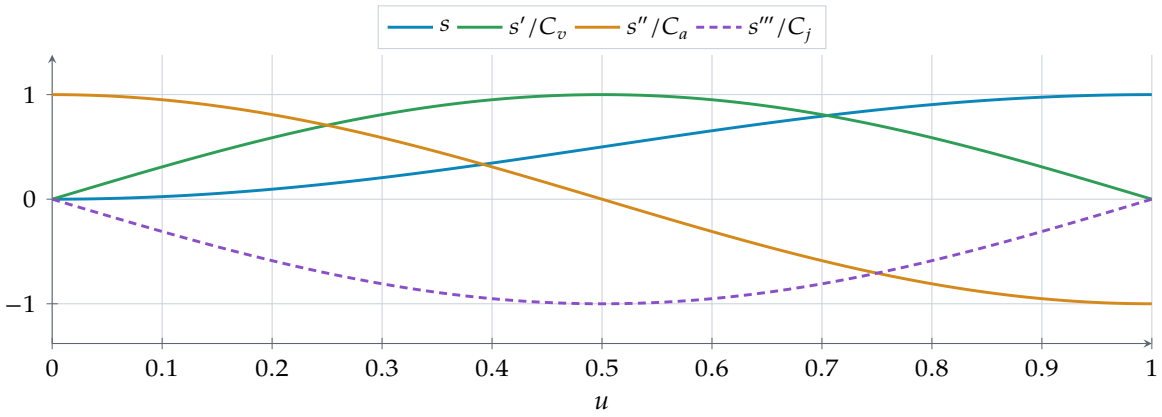


Figure 3.1. Simple harmonic. The acceleration does not go to zero at the ends.

3.5 Cycloidal

The cycloidal law has a full sine wave of acceleration:

$$s = u - \frac{\sin 2\pi u}{2\pi}, \quad s' = 1 - \cos 2\pi u, \quad s'' = 2\pi \sin 2\pi u, \quad s''' = 4\pi^2 \cos 2\pi u. \tag{3.4}$$

The velocity and the acceleration are zero at both ends. Thus the law joins a dwell without a step in velocity or acceleration. The jerk is $4\pi^2$ at both ends, so the jerk steps where the law meets a dwell.

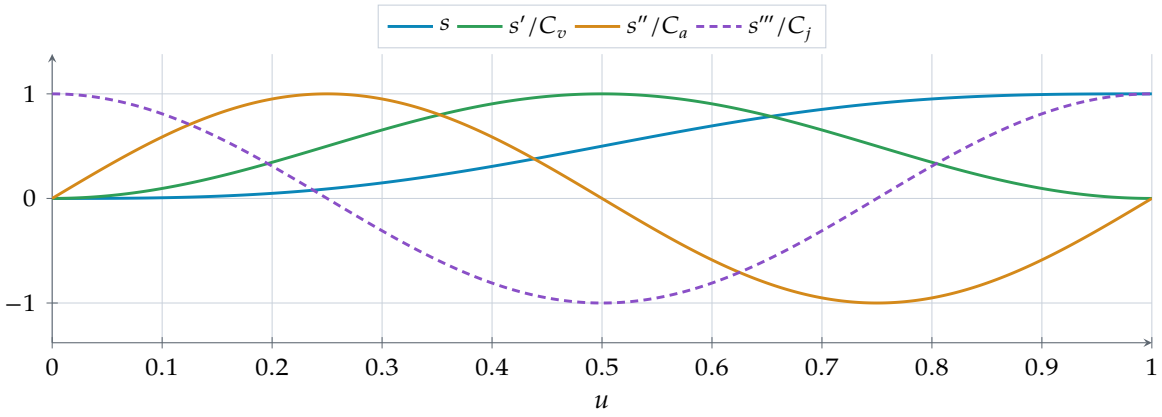


Figure 3.2. Cycloidal. The acceleration is zero at both ends.

3.6 Piecewise Sinusoidal Laws

The modified trapezoid and the modified sine are made from pieces. On each piece, from u_a to u_b, the raw acceleration is

$$a(\tau) = c + k\sin(w\tau + \varphi), \qquad \tau = u - u_a. \tag{3.5}$$

The code integrates each piece in closed form:

$$j(\tau) = kw\cos(w\tau + \varphi), \tag{3.6}$$

$$v(\tau) = v_a + c\tau + \frac{k}{w}(\cos\varphi - \cos(w\tau + \varphi)), \tag{3.7}$$

$$s(\tau) = s_a + v_a\tau + \tfrac{1}{2}c\tau^2 + \frac{k}{w}\tau\cos\varphi - \frac{k}{w^2}(\sin(w\tau + \varphi) - \sin\varphi). \tag{3.8}$$

Each piece starts with the values v_a and s_a at the end of the piece before it. The code then multiplies s, s', s'' and s''' by one factor, so that $s(1) = 1$. The raw acceleration of each piece has a peak of 1, so the factor is C_a.

3.6.1 Modified Trapezoid

The modified trapezoid has five pieces, with $w = 4\pi$ and $C_a = 8\pi/(\pi + 2)$:

$$s''(u) = C_a \times \begin{cases} \sin 4\pi u & 0 \le u \le 1/8 \\ 1 & 1/8 \le u \le 3/8 \\ \cos 4\pi(u - 3/8) & 3/8 \le u \le 5/8 \\ -1 & 5/8 \le u \le 7/8 \\ -\cos 4\pi(u - 7/8) & 7/8 \le u \le 1 \end{cases} \tag{3.9}$$

The acceleration is constant for half of the segment. Thus the law has the lowest peak acceleration of the common laws. The jerk is continuous inside the law: it is zero on both sides of each join.

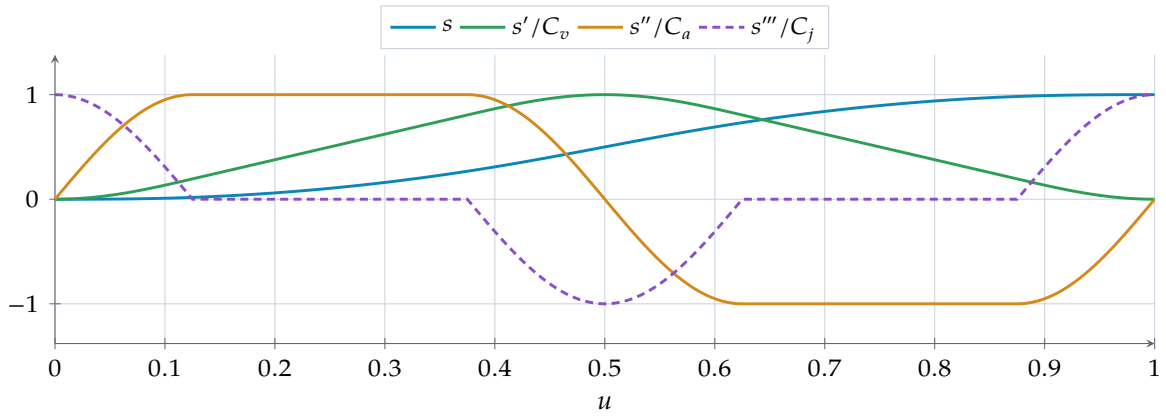


Figure 3.3. Modified trapezoid. The flat tops give the low peak acceleration.

3.6.2 Modified Sine

The modified sine has three pieces, with $C_a = 4\pi^2/(\pi + 4)$:

$$s''(u) = C_a \times \begin{cases} \sin 4\pi u & 0 \le u \le 1/8 \\ \cos \frac{4\pi}{3}(u - 1/8) & 1/8 \le u \le 7/8 \\ -\cos 4\pi(u - 7/8) & 7/8 \le u \le 1 \end{cases} \tag{3.10}$$

The long middle piece gives the lowest peak velocity of the laws that start and end at rest with zero acceleration: $C_v = 4\pi/(\pi + 4)$. A low peak velocity gives a low peak motor speed.

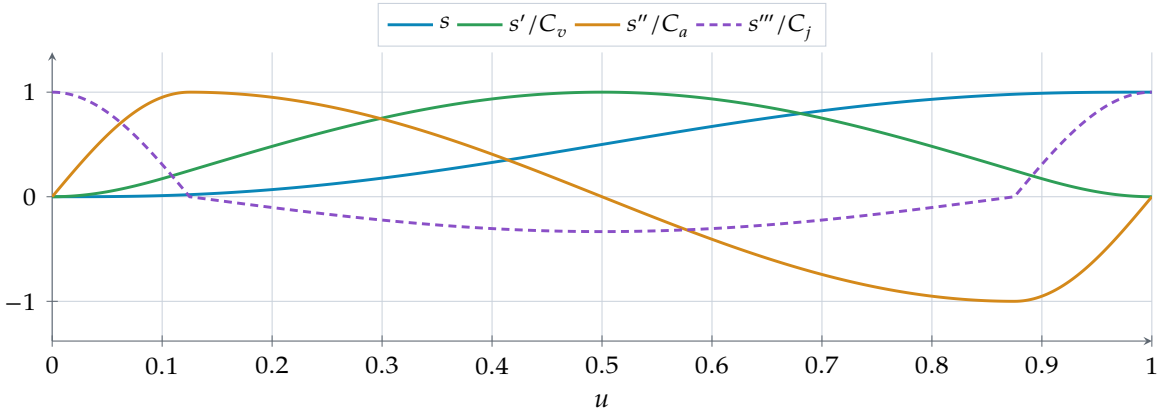


Figure 3.4. Modified sine. The low velocity peak gives a low peak motor speed.

3.7 Polynomial Laws

3.7.1 Polynomial 3-4-5

The 3-4-5 polynomial is the lowest-order polynomial that gives zero velocity and zero acceleration at both ends:

$$s = 10u^3 - 15u^4 + 6u^5, \qquad s' = 30u^2 - 60u^3 + 30u^4, \tag{3.11}$$

$$s'' = 60u - 180u^2 + 120u^3, \qquad s''' = 60 - 360u + 360u^2. \tag{3.12}$$

The jerk is 60 at both ends.

3.7.2 Polynomial 4-5-6-7

The 4-5-6-7 polynomial also gives zero jerk at both ends:

$$s = 35u^4 - 84u^5 + 70u^6 - 20u^7, \qquad s' = 140u^3 - 420u^4 + 420u^5 - 140u^6, \tag{3.13}$$

$$s'' = 420u^2 - 1680u^3 + 2100u^4 - 840u^5, \qquad s''' = 840u - 5040u^2 + 8400u^3 - 4200u^4. \tag{3.14}$$

The cost of the zero end jerk is a high peak acceleration: $C_a = 84\sqrt{5}/25 \approx 7.51$.

3.8 Blend and Cubic Spline

Blend and the cubic spline depend on their neighbours. Chapter 5 gives them. Without moving neighbours, a Blend is the 3-4-5 polynomial, and a single spline segment is $s = 3u^2 - 2u^3$.

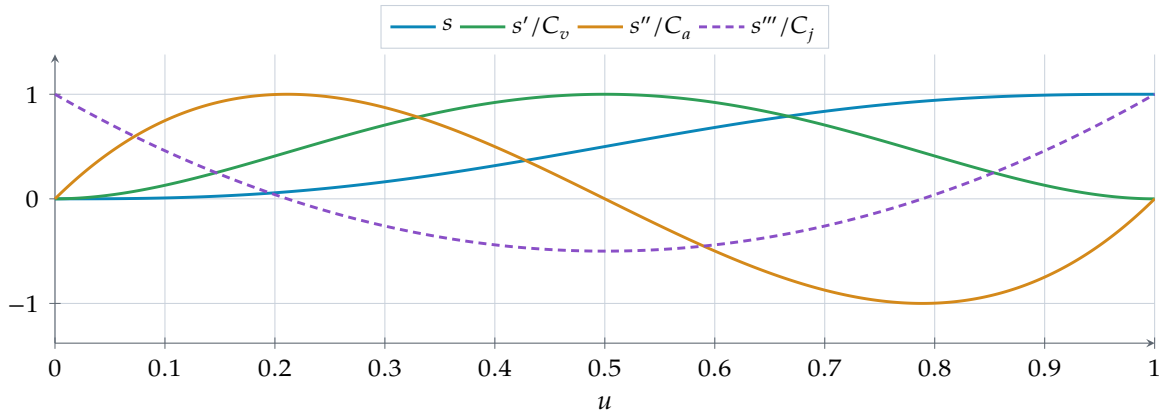


Figure 3.5. Polynomial 3-4-5.

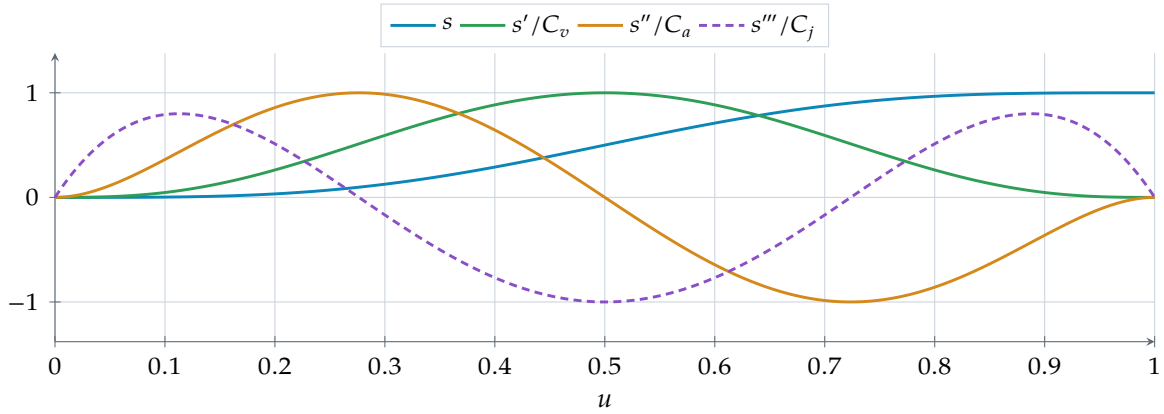


Figure 3.6. Polynomial 4-5-6-7. The jerk is zero at both ends.

3.9 Comparison of the Laws

Figure 3.7 shows the acceleration of each law that starts and ends at rest. Table 3.1 gives the peak coefficients. The values for C_v and C_a agree with the published values that the test `test_camlaws` checks. The values for C_j come from the code, sampled at 200,001 points.

3.10 Selection of a Law

Use the table and these rules:

- For a heavy load, use the modified trapezoid. It has the lowest peak acceleration, so it has the lowest peak force.
- For a motor near its speed limit, use the modified sine. It has the lowest peak velocity of the laws with zero end acceleration.
- For a general rise or return from a dwell, use the cycloidal law or the 3-4-5 polynomial.
- For the least vibration in a light and fast mechanism, use the 4-5-6-7 polynomial. It also has zero jerk at its ends.
- Do not use the linear law or the simple harmonic law next to a dwell.

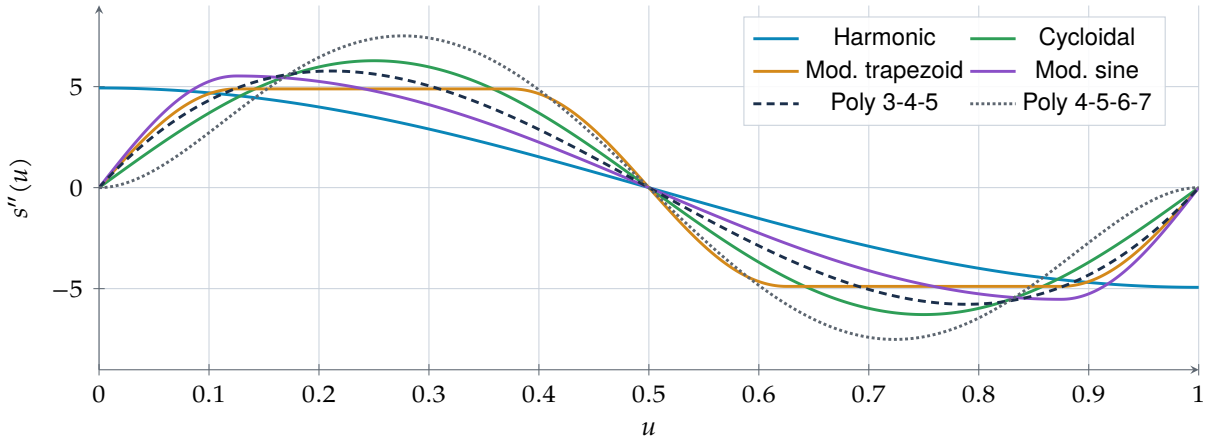


Figure 3.7. The acceleration of the laws for the same rise and span.

Table 3.1. The peak coefficients and end conditions of the laws.

Law	C_v	C_a	C_j	**At the ends**
Linear	1	0	0	velocity step
Simple harmonic	$\pi/2 = 1.571$	$\pi^2/2 = 4.935$	$\pi^3/2 = 15.50$	acceleration step
Cycloidal	2	$2\pi = 6.283$	$4\pi^2 = 39.48$	$s' = s'' = 0$
Modified trapezoid	2	$8\pi/(\pi+2) = 4.888$	$32\pi^2/(\pi+2) = 61.43$	$s' = s'' = 0$
Modified sine	$4\pi/(\pi+4) = 1.760$	$4\pi^2/(\pi+4) = 5.528$	$16\pi^3/(\pi+4) = 69.47$	$s' = s'' = 0$
Polynomial 3-4-5	$15/8 = 1.875$	$10/\sqrt{3} = 5.774$	60	$s' = s'' = 0$
Polynomial 4-5-6-7	$35/16 = 2.188$	$84\sqrt{5}/25 = 7.513$	52.5	$s' = s'' = s''' = 0$

NOTE: A longer span always helps more than a better law. Equation (2.6) divides the acceleration by β^2, so a segment that is 20 % longer needs about 31 % less peak acceleration.

Chapter 4

Building a Profile

This chapter puts segments together into the profile of an axis. It gives the rules that keep the profile valid, how the code finds the position at a master position, and the electronic gear.

4.1 The Data of an Axis

An axis has:

- a start position y_{start}, the position at knot 0;
- a list of segments, each with its end on the master x_1, its end position y_1 and its law;
- a span X: 360° on a crank, the duration on a timeline;
- a *closed* flag, which makes the profile end at its start position.

Knot k for $k \geq 1$ is the end of segment $k - 1$. Segment i starts at knot i. Thus the start position of a segment is always the end position of the segment before it.

4.2 Ties Between Knots

Some knots must have the same position. The code keeps groups of tied knots. Each knot takes the position of the first knot in its group:

- A dwell ties its two knots.
- A closed profile ties knot 0 and the last knot.

When you move a knot, all the knots in its group move. A closed profile only makes the last position equal to the first position. It does not make the velocity or the acceleration continuous across the end of the cycle. The analysis checks that (section 4.6).

4.3 Repeating and One-Shot Axes

An axis is *cyclic* when its profile repeats:

- An axis on a crank is always cyclic.
- An axis on a timeline is cyclic only when the timeline repeats and the span of the axis is the full cycle.

The cycle of a timeline is the largest span of its axes and of its output pulses. A one-shot axis is at rest before $x = 0$, at its start position. It is at rest after its span, at its last position.

An *open* profile ends at a different position from its start. On a crank, the axis then advances by

$$\Delta = y_{\text{end}} - y_{\text{start}} \tag{4.1}$$

each revolution. An index feed uses this: the feed advances 50 mm each cycle and never returns.

4.4 Finding the Position

The function `Evaluate` gives the position at a master position x:

1. On a cyclic axis, it wraps x into $0 \le x < X$. On a one-shot axis, it returns the rest position outside $0 \le x \le X$.
2. It finds the first segment with $x < x_1$. The segments are half-open intervals $[x_0, x_1)$, so at a knot the code uses the segment that starts at the knot.
3. It calculates $u = (x - x_0)/\beta$ with equation (2.4) and evaluates the law.

4.5 Rules That Keep a Profile Valid

Before the code uses a profile, it makes the profile valid:

- An axis without segments gets one dwell.
- Each segment is at least $\beta_{\min} = 0.5 \times X/360$ long: 0.5° on a crank.
- The last segment ends at X.
- The tied knots take the position of their group.

An axis has at most 64 segments. Thus a profile has at most 65 knots.

4.6 Continuity at the Knots

Each law is smooth inside its segment. A problem can only occur at a knot, where two laws meet. Figure 4.1 shows the carton pusher. Its rise is a modified sine, and its return is a 4-5-6-7 polynomial. Both laws meet the dwells with zero velocity and zero acceleration, so the velocity and the acceleration are continuous at each knot.

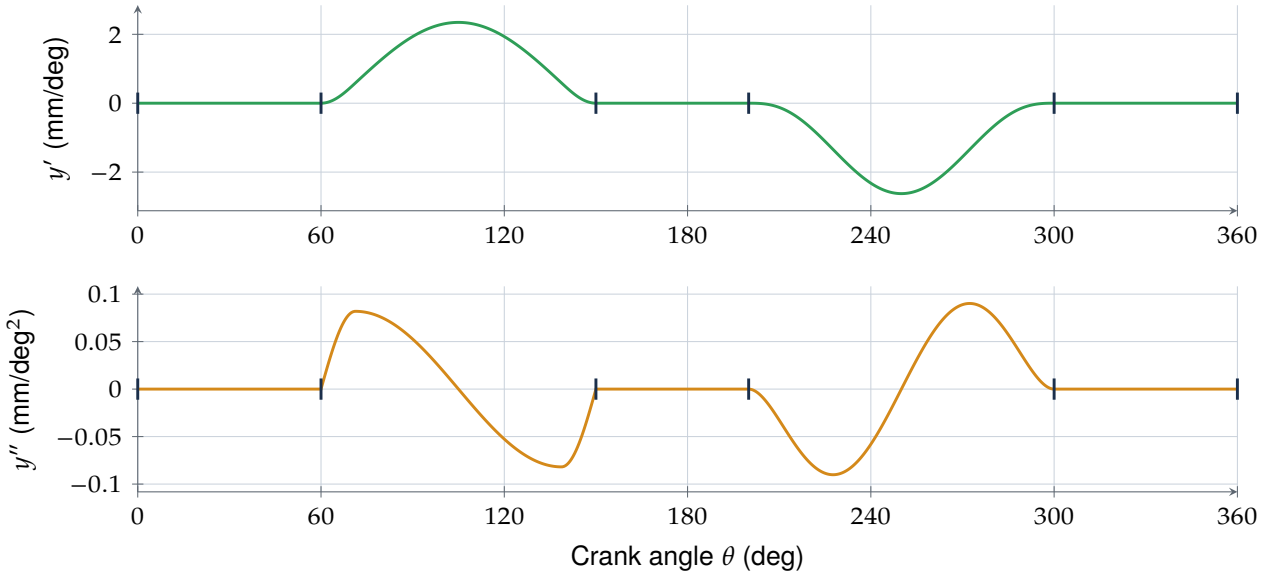


Figure 4.1. The velocity and acceleration of the carton pusher, per degree of the crank. The marks show the knots.

The analysis compares the two sides of each knot. On a cyclic axis, it also compares the end

of the cycle with its start. On a one-shot axis, it compares the ends with rest. With L the value at the end of the left segment and R the value at the start of the right segment:

$$\text{velocity step:} \quad |y'_L - y'_R| > 10^{-6} \max(|y'_L|, |y'_R|) + 10^{-9}, \tag{4.2}$$

$$\text{acceleration step:} \quad |y''_L - y''_R| > 10^{-6} \max(|y''_L|, |y''_R|) + 10^{-12}. \tag{4.3}$$

A velocity step is an error. An acceleration step is a warning. At a knot with a velocity step, the analysis does not also report the acceleration step.

4.7 The Electronic Gear

An electronic gear is an axis that follows the crank at a fixed ratio n_g/d_g axis units for each revolution. The code makes it one open linear segment from 0° to 360°, with a rise of n_g/d_g. A negative n_g turns the axis the other way.

A gear must not drift, even after millions of revolutions. Thus the firmware does not use the linear law at run time. It follows the master in whole numbers. With c the counts per unit of the axis and U the master units for each revolution (chapter 11), the gear ratio in motor counts for each master unit is

$$\frac{P}{Q} = \frac{c\, n_g}{d_g\, U}. \tag{4.4}$$

If $c\, n_g$ is not a whole number, the firmware doubles both terms until it is, or until it reaches 2^{38}. Then it rounds both. Each sample, with m the whole master units that the master moved:

$$A \leftarrow A + m\, P, \tag{4.5}$$

$$d = \lfloor A/Q \rfloor, \qquad A \leftarrow A - d\, Q, \tag{4.6}$$

$$y_{\text{counts}} \leftarrow y_{\text{counts}} + d. \tag{4.7}$$

The remainder A stays in $0 \le A < Q$. Thus the position is exactly $\lfloor MP/Q \rfloor$ counts after the master has moved M units, at all times.

> An electronic gear is exact in whole numbers. Its error does not grow with time.

This page is intentionally left blank.

Chapter 5

Blend and the Cubic Spline

Two laws take their shape from their neighbours. Blend joins two motions that are not at rest. The cubic spline makes a smooth curve through a list of points. This chapter gives both.

5.1 Blend

5.1.1 The Problem

A flying saw must match the speed of the material before it cuts. The carriage waits, accelerates to the line speed, moves at the line speed, decelerates and returns. The part at the line speed is a linear segment. The segments before and after it must start or end at the velocity of the linear segment. A law from chapter 3 cannot do this, because each of those laws starts and ends at rest.

5.1.2 The Construction

Blend is a quintic Hermite polynomial. It has six conditions:

- the position goes from y_0 to $y_0 + h$;
- the velocity goes from the velocity v_0 of the left neighbour to the velocity v_1 of the right neighbour;
- the acceleration is zero at both ends.

The code first changes the end velocities to the unit square: $\hat{v}_0 = v_0\beta$ and $\hat{v}_1 = v_1\beta$. Then

$$y(u) = y_0 + h\,P(u) + \hat{v}_0\,B_0(u) + \hat{v}_1\,B_1(u), \tag{5.1}$$

with the three basis polynomials

$$P(u) = 10u^3 - 15u^4 + 6u^5, \tag{5.2}$$

$$B_0(u) = u - 6u^3 + 8u^4 - 3u^5, \tag{5.3}$$

$$B_1(u) = -4u^3 + 7u^4 - 3u^5. \tag{5.4}$$

P moves the position and has zero velocity and acceleration at both ends. B_0 has a velocity of 1 at $u = 0$ and of 0 at $u = 1$. B_1 has the opposite. All three have zero acceleration at both ends. The code divides the derivatives by β, β^2 and β^3, as in equation (2.6).

5.1.3 The Neighbours

The velocity of a neighbour is its velocity at the shared knot. These rules apply:

- Past the end of a one-shot axis, the velocity is zero.
- On a cyclic axis, the neighbour of the last segment is the first segment.
- A neighbour that is also a Blend counts as at rest. Thus two Blends never depend on each other.
- On an axis with one segment, the velocity is zero.

When both neighbours are at rest, $\hat{v}_0 = \hat{v}_1 = 0$ and Blend is the 3-4-5 polynomial.

CAUTION

Blend has zero acceleration at its ends. A neighbour with an acceleration at the shared knot, for example a simple harmonic law or a clamped spline end, gives an acceleration step there.

5.1.4 An Example: The Carriage of a Flying Saw

Figure 5.1 shows the carriage of the example `variable_length_cutoff`. A Blend accelerates the carriage from rest to the line speed. The linear segment holds the line speed through the sync, where the saw cuts. A second Blend decelerates it. A 3-4-5 polynomial returns it home.

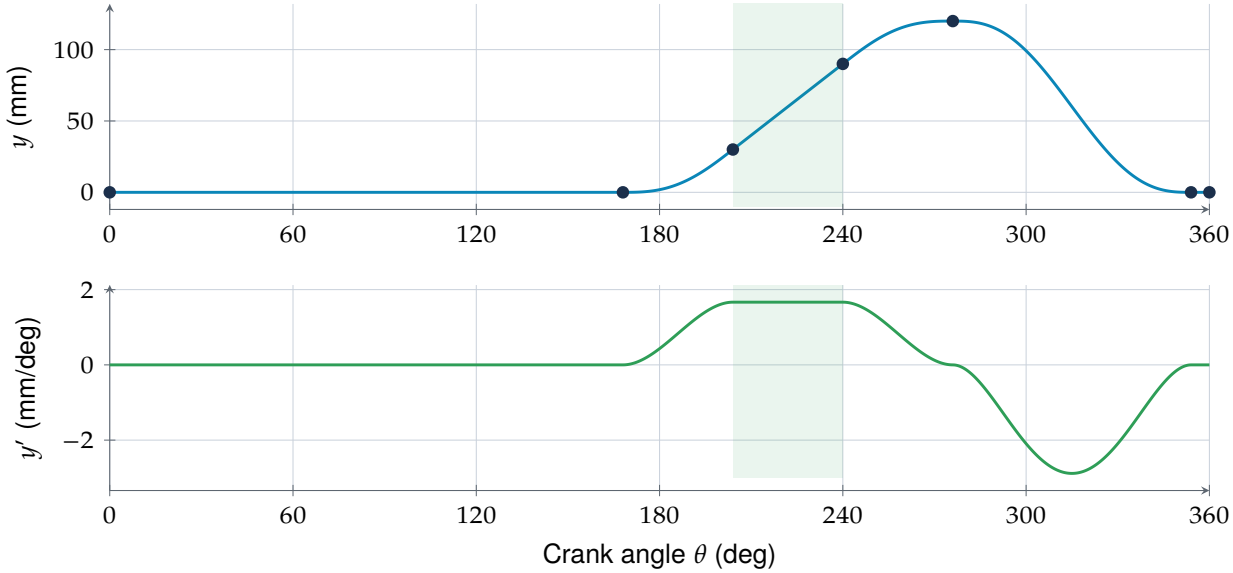


Figure 5.1. The carriage of a flying saw. The shaded band is the linear segment: the sync. The Blends meet it at its velocity.

5.2 The Cubic Spline

5.2.1 The Model

A run of consecutive spline segments is one curve: a cubic polynomial on each segment, with continuous position, velocity and acceleration at each knot. The curve goes through each knot (x_i, y_i). This is a C^2 cubic spline.

Let M_i be the acceleration y'' at knot i. On a segment of width w from knot i to knot $i+1$, let $a = w(1-u)$ be the distance to the end and $b = uw$ the distance from the start. Then

$$y = \frac{M_i a^3 + M_{i+1} b^3}{6w} + c_0 a + c_1 b, \qquad c_0 = \frac{y_i}{w} - \frac{M_i w}{6}, \quad c_1 = \frac{y_{i+1}}{w} - \frac{M_{i+1} w}{6}, \tag{5.5}$$

$$y' = \frac{M_{i+1} b^2 - M_i a^2}{2w} + c_1 - c_0, \qquad y'' = \frac{M_i a + M_{i+1} b}{w}, \qquad y''' = \frac{M_{i+1} - M_i}{w}. \tag{5.6}$$

These are derivatives with respect to the master directly. The acceleration is linear on each segment, and the jerk is constant.

5.2.2 The Equations

The acceleration must be continuous at each inner knot. With h_j the width of segment j and $d_j = (y_{j+1} - y_j)/h_j$ its slope, each inner knot i gives one equation:

$$\mu_i M_{i-1} + 2M_i + \lambda_i M_{i+1} = \frac{6\,(d_i - d_{i-1})}{h_{i-1} + h_i}, \qquad \mu_i = \frac{h_{i-1}}{h_{i-1} + h_i}, \quad \lambda_i = \frac{h_i}{h_{i-1} + h_i}. \tag{5.7}$$

The ends of the run give two more equations:

- **Clamped.** The velocity at the end is a known value v: $2M_0 + M_1 = (6/h_0)(d_0 - v)$ at the start, or $M_{n-1} + 2M_n = (6/h_{n-1})(v - d_{n-1})$ at the end.
- **Natural.** The acceleration at the end is zero: $M = 0$.

The code selects the end condition from the neighbour of the run:

- Next to a Blend, the end is natural.
- Next to any other law, the end is clamped to the velocity of that neighbour. A dwell gives a velocity of zero.
- At the start and the end of a one-shot axis, the end is clamped to zero velocity.

The equations make a tridiagonal system. Each row has $|2| > |\mu_i| + |\lambda_i| = 1$, so the matrix is strictly diagonally dominant. The Thomas algorithm solves it in $O(n)$ steps without pivoting.

5.2.3 The Periodic Spline

On a cyclic axis where every segment is a spline, the curve has no ends. The last knot is knot 0 again. Equation (5.7) then applies at every knot, with the indices taken modulo n. The matrix is tridiagonal except for two corner terms. The code solves it with the Sherman–Morrison formula: it removes the corners, solves two tridiagonal systems, and corrects the result. For $n = 2$ it solves the two equations directly.

Figure 5.2 shows the example `bench_spline`: a periodic spline on a crank. The acceleration is continuous and piecewise linear.

5.2.4 Fitting Imported Points

The Workbench can import a list of points as a spline. A profile has at most 65 knots. With more points than that, the code selects the knots one at a time. It starts with the two end points. Then it adds the point that is farthest from the current spline, until it has 65 knots. A profile that ends at its start position, within 10^{-9} of its range, is closed.

CAUTION

A spline can overshoot between its knots. The analysis gives a warning when the curve goes more than 2 % of the knot range past the highest or lowest knot.

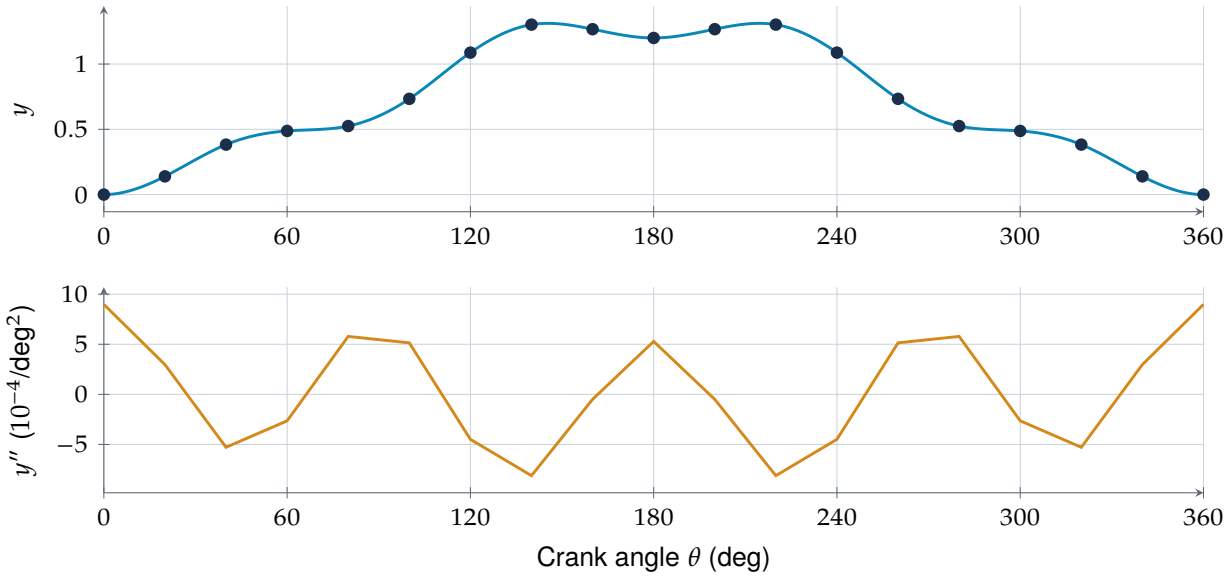


Figure 5.2. A periodic cubic spline through its knots, and its acceleration: continuous and linear between the knots.

Chapter 6

Analysis and Speed Limits

A profile can be smooth and still be too fast for the machine. This chapter gives the values that the analysis calculates, how it converts them to motor values, and how it finds the fastest safe speed of the machine.

6.1 Sampling

The analysis samples each axis at N equal steps of the master: $x_i = X\,i/N$, with $N = 3600$. At each sample it evaluates y, y', y'' and y'''. It also includes the start and end positions, so the stroke of an open profile includes its advance. The speed-limit calculation uses $N = 1440$.

6.2 The Peak Values

The analysis records the largest $|y'|$, $|y''|$ and $|y'''|$, and the first master position of each. It converts them to time with equation (2.7):

$$v_{\max} = \max|y'|\,\omega, \qquad a_{\max} = \max|y''|\,\omega^2, \qquad j_{\max} = \max|y'''|\,\omega^3. \tag{6.1}$$

6.3 From Axis Units to the Motor

6.3.1 Counts per Unit

Each axis has c, the motor counts (steps) for one axis unit. The drive type gives c from the mechanics: A gearbox is not a separate term. Include its ratio in p_m.

Table 6.1. Counts per unit for each drive type

Drive	Counts per unit	Data
Direct	c as you enter it	–
Ball screw	$c = p_m/P$	p_m motor counts per revolution, P screw lead
Belt	$c = p_m/(\pi d)$	d pitch diameter of the pulley

6.3.2 Step Rate and Motor Speed

The peak step rate and the peak motor speed are

$$f_{\text{step}} = \max|y'|\;\omega\;c \quad \text{counts/s}, \qquad n_{\text{motor}} = \frac{60\,f_{\text{step}}}{p_m} \quad \text{rpm}. \tag{6.2}$$

6.3.3 The Position Table

At run time, the ClearCore does not evaluate the laws. It uses a table of whole motor counts for each axis (section 12.1). Each entry is

$$T[k] = \text{round}(y(X\,k/K) \cdot c), \qquad k = 0, 1, \ldots, K, \qquad K = 2048. \tag{6.3}$$

6.4 An Example: The Carton Pusher

The carton pusher has a ball screw with a 10 mm lead and a motor with 800 counts per revolution, so $c = 80$ counts/mm. The crank turns at 32 rpm, so $\omega = 192\,^\circ\,\text{s}^{-1}$. The peak velocity comes

Table 6.2. The carton pusher at 32 rpm

Value	Per degree	At 32 rpm	Axis limit
Peak velocity	2.625 mm/deg	504 mm s^{-1}	600 mm s^{-1}
Peak acceleration	0.090, 2 mm/deg^2	3,324 mm s^{-2}	8,000 mm s^{-2}
Peak step rate	–	40,320 counts/s	–
Peak motor speed	–	3,024 rpm	–

from the 4-5-6-7 return: $C_v h/\beta = 2.1875 \times 120/100 = 2.625$ mm/deg. The modified-sine rise is slower: $1.7596 \times 120/90 = 2.346$ mm/deg. The peak acceleration also comes from the return: $7.513 \times 120/100^2 = 0.090, 2$ mm/deg^2.

6.5 The Fastest Safe Speed

Each axis has a velocity limit V and an acceleration limit A in axis units per second. At each sample i, the master speed must obey two conditions:

$$|y_i'|\,\omega \le V, \qquad |y_i''|\,\omega^2 + |y_i'|\,\alpha \le A. \tag{6.4}$$

The second condition is equation (2.8) with both terms at their worst sign. Solve each condition for ω:

$$\omega_{\max} = \min_i \; \min\left(\frac{V}{|y_i'|},\; \sqrt{\frac{A - |y_i'|\,\alpha}{|y_i''|}} \right). \tag{6.5}$$

If $A - |y_i'|\,\alpha \le 0$ at a sample, the ramp alone uses all the acceleration, and $\omega_{\max} = 0$. The ramp that the axis can accept at all is

$$\alpha_{\max} = \frac{A}{\max |y'|}. \tag{6.6}$$

The speed limit of the machine is the smallest limit of its axes. On a driven crank, the limit of the crank motor also applies:

$$N_{\max} = \min\left(N_{\text{crank}},\; \min_{\text{axes}} \frac{\omega_{\max}}{6} \right) \quad \text{rpm}. \tag{6.7}$$

On a timeline, $\alpha = 0$ and $\omega_{\max}$ is the largest speed override.

Figure 6.1 applies equation (6.5) to the carton pusher. The crank ramps at 20 rpm/s, so $\alpha = 120\,^\circ\,\text{s}^{-2}$. Each curve is the limit at one crank angle. The lowest point of the two curves is the limit of the axis: 38.1 rpm, set by the velocity on the return. The acceleration alone allows 49.1 rpm.

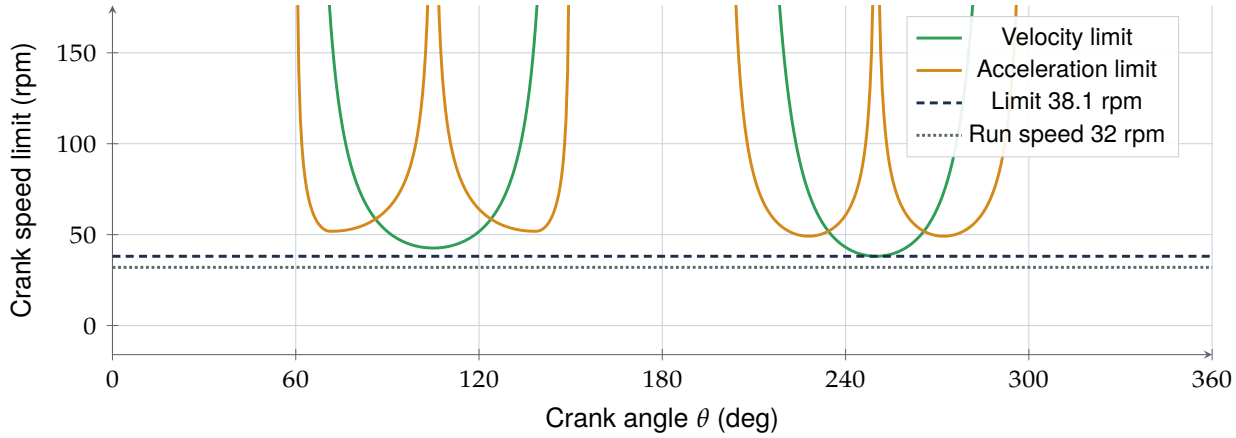


Figure 6.1. The speed limit of the carton pusher at each crank angle. In the dwells, the axis does not move and has no limit.

6.6 The Checks

The analysis reports these conditions:

Table 6.3. The checks of the analysis

Check	Condition	Level
Velocity step	A step in y' at a knot (section 4.6)	Error
Acceleration step	A step in y'' at a knot	Warning
Spline overshoot	A spline goes past its highest or lowest knot by more than 2 % of the knot range	Warning
Pressure angle	A plate cam has $\phi > 30°$ (chapter 7)	Warning
Undercut	A plate cam has $\rho_{\min} < R_r$	Warning
Velocity limit	$\max \lvert y' \rvert \omega > V$	Error
Acceleration limit	$\max(\lvert y'' \rvert \omega^2 + \lvert y' \rvert \alpha) > A$	Error
Speed	The run speed or the override is more than the limit	Error
Connector	Two axes, or an axis and the crank motor, use one motor connector	Error

NOTE: The velocity check uses 3600 samples. The speed limit uses 1440 samples. Thus a speed just below the limit can still give a velocity warning. Keep a small margin.

CAUTION

A limit switch stops a driven crank in the limit stop time, 0.1 s by default. This stop can be steeper than the ramp that the speed limit uses. Set the limit stop time for the mechanics.

This page is intentionally left blank.

Chapter 7

Plate Cam Geometry

An axis on a crank can also be cut as a plate cam. The Workbench draws the cam, exports it as a DXF file and checks its geometry. The geometry does not change the motion of the ClearCore. It shows whether a mechanical cam with the same profile is possible, and it shows how hard the profile pushes the follower.

7.1 The Frame

The origin is the center of the cam. At $\theta = 0$ the follower is at the top of the cam (12 o'clock). The cam turns clockwise. The follower is a roller of radius R_r that moves along a radial line. It has no offset. With $R_r = 0$ it is a knife edge. The base radius R_b is the radius of the cam surface at the lowest position of the follower.

7.2 The Pitch Curve

The pitch curve is the path of the center of the roller. At crank angle θ, its point is at the polar angle $\varphi = 90° + \theta$, at the radius

$$R(\theta) = R_b + R_r + (y(\theta) - y_{\min}). \tag{7.1}$$

The derivatives with respect to the polar angle in radians are

$$R' = y' \frac{180}{\pi}, \qquad R'' = y'' \left(\frac{180}{\pi}\right)^2, \tag{7.2}$$

because y' and y'' are per degree.

7.3 The Cam Surface

The surface is the inner envelope of all the positions of the roller. For each ray from the center, at the polar angle φ_i with the unit vector $\hat{u}$, the code finds where the ray first enters a roller circle. For a circle at the pitch point P_j:

$$a = P_j \cdot \hat{u}, \qquad b = P_{j,x}\hat{u}_y - P_{j,y}\hat{u}_x, \qquad t_j = a - \sqrt{R_r^2 - b^2}. \tag{7.3}$$

The circle counts only if $a > 0$ and $R_r^2 \geq b^2$. The surface radius on the ray is the smallest t_j. The code tests only the circles near the ray: those within $\arcsin(R_r/(R_b + R_r))$ of it, plus two samples.

Where the profile is smooth, the envelope is the pitch curve moved inward by R_r. At a sharp corner, the envelope also trims the inside of the corner and rounds the outside, as a real roller does.

Figure 7.1 shows the carton pusher as a plate cam, with $R_b = 170\,\text{mm}$ and $R_r = 14\,\text{mm}$.

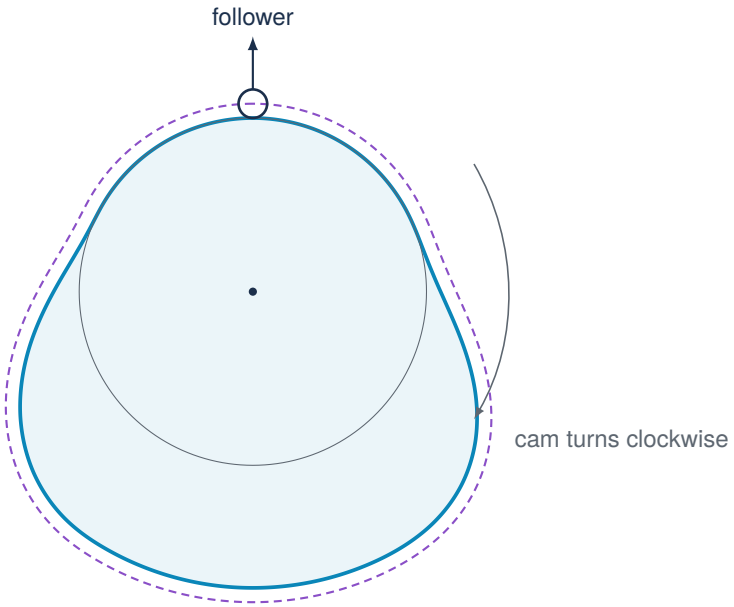


Figure 7.1. The carton pusher as a plate cam. Solid: the cam surface. Dashed: the pitch curve. Thin: the base circle. The roller is shown at $\theta = 0$.

7.4 The Pressure Angle

The pressure angle ϕ is the angle between the direction of motion of the follower and the normal to the cam at the contact point. A large pressure angle pushes the follower sideways in its guide. The code calculates

$$\tan\phi = \frac{|R'|}{R} = \frac{|y'|\,(180/\pi)}{R_b + R_r + y - y_{\min}}. \tag{7.4}$$

The analysis gives a warning when $\phi > 30°$. To decrease the pressure angle, make the base circle larger or make the segment longer.

Figure 7.2 shows the pressure angle of the carton pusher, with its sign: negative on the rise and positive on the return. The largest value is 32.3° at 254.6°, on the return. Thus the analysis gives a warning for this cam.

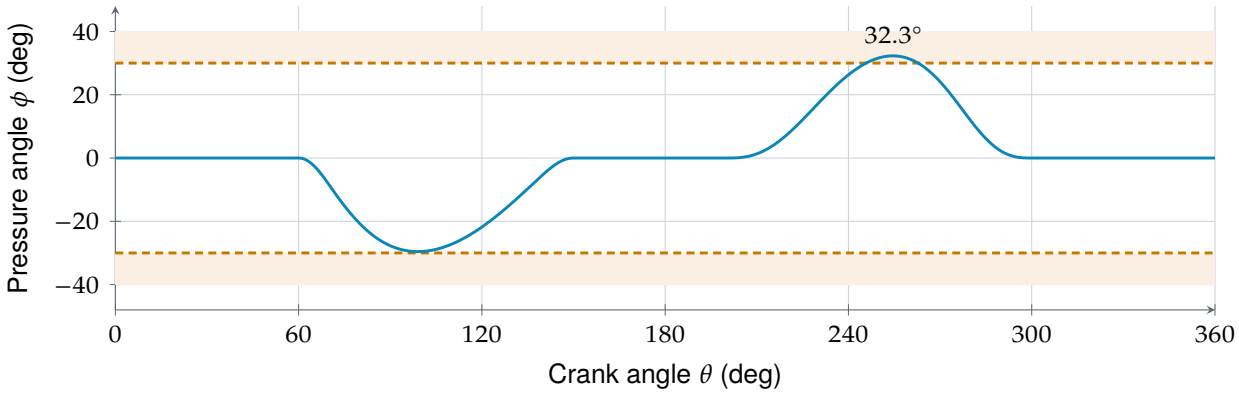


Figure 7.2. The pressure angle of the carton pusher. The shaded bands are above the 30° limit.

7.5 Curvature and Undercut

The radius of curvature of the pitch curve is

$$\rho = \frac{\left(R^2 + R'^2\right)^{3/2}}{R^2 + 2R'^2 - R\,R''}. \tag{7.5}$$

Where the denominator is positive, the pitch curve is convex. The surface there has the radius $\rho - R_r$. If $\rho < R_r$, the radius of the surface is negative: the roller cannot follow the curve, and the cutter removes material that the profile needs. This is *undercut*. The analysis gives a warning when the smallest convex ρ is less than R_r.

The carton pusher has $\rho_{\min} = 150.1\,\text{mm}$ at 228.6°. This is much larger than $R_r = 14\,\text{mm}$, so the cam has no undercut.

> **NOTE:** The pressure angle and the curvature apply only to an axis on a crank. An open profile has a radial step at 0°, where the roller drops through to the start position.

This page is intentionally left blank.

Chapter 8

Cut-Length Mode

A flying saw cuts a moving line of material into pieces. The length of the piece can change during production, but the profile is drawn for one length. Cut-length mode stretches the cycle to the length of the piece. It does not change the motion of the carriage or the saw.

8.1 The Terms

- q is the line counts per millimetre. An encoder on a measuring wheel gives $q = p/W$, with p the counts per revolution and W the circumference of the wheel.
- D_{mm} is the design length: the line travel for one cycle as drawn. The cycle has $D = \text{round}(D_{\text{mm}}\, q)$ counts. The gear of the master is 1:1.
- L is the length of the finished piece and k_f is the kerf: the material that the saw removes. The cycle for a piece of length L has $C = \text{round}((L + k_f)\, q)$ counts.
- One degree on the drawing is $D_{\text{mm}}/360$ millimetres of line.

8.2 The Wait at the Start of the Cycle

The drawn cycle must start with a wait, where every axis on the crank dwells. Its length W_{deg} is the smallest of:

- the end of the first run of dwells on each axis on the crank (gear axes are not included), and
- the start of each output pulse on the crank.

In counts, $W = \text{round}(W_{\text{deg}}/360 \cdot D)$. The wait must be longer than zero.

8.3 Stretching the Cycle

Let φ be the line position in the current cycle, $0 \le \varphi < C$. The drawn position is

$$x = \max(0,\ \varphi - (C - D)), \qquad \theta = 360\,\frac{x}{D}. \tag{8.1}$$

For a long piece, $C > D$: the wait is longer by $C - D$, and the rest of the cycle is as drawn. For a short piece, $C < D$: the wait is shorter. Thus each motion after the wait occurs at the same line speed as drawn, and the carriage meets the material at the correct speed for every length.

The shortest piece makes the wait zero, $C = D - W$:

$$L_{\text{min}} = \frac{D - W}{q} - k_f. \tag{8.2}$$

There is no longest piece.

8.4 An Example: The Variable-Length Cutoff

The example `variable_length_cutoff` has a 200 mm wheel and an encoder with 4000 counts per revolution, so $q = 20$ counts/mm. The design length is 600 mm, so $D = 12000$ counts. The kerf is 3 mm. The carriage dwells until 168°, so $W_{\text{deg}} = 168°$ and $W = 5600$ counts. The shortest piece is $(12000 - 5600)/20 - 3 = 317$ mm. The piece as drawn is $600 - 3 = 597$ mm.

The sync is the linear segment of the carriage, from 204° to 240°. It moves 60 mm in 36°: 1.667 mm/deg. The line moves 600/360 = 1.667 mm/deg. Thus the carriage matches the line during the sync.

Figure 8.1 shows the carriage and the saw plunge against the line travel for the piece as drawn and for a piece of 897 mm ($C - D = 300$ mm). Only the wait is longer.

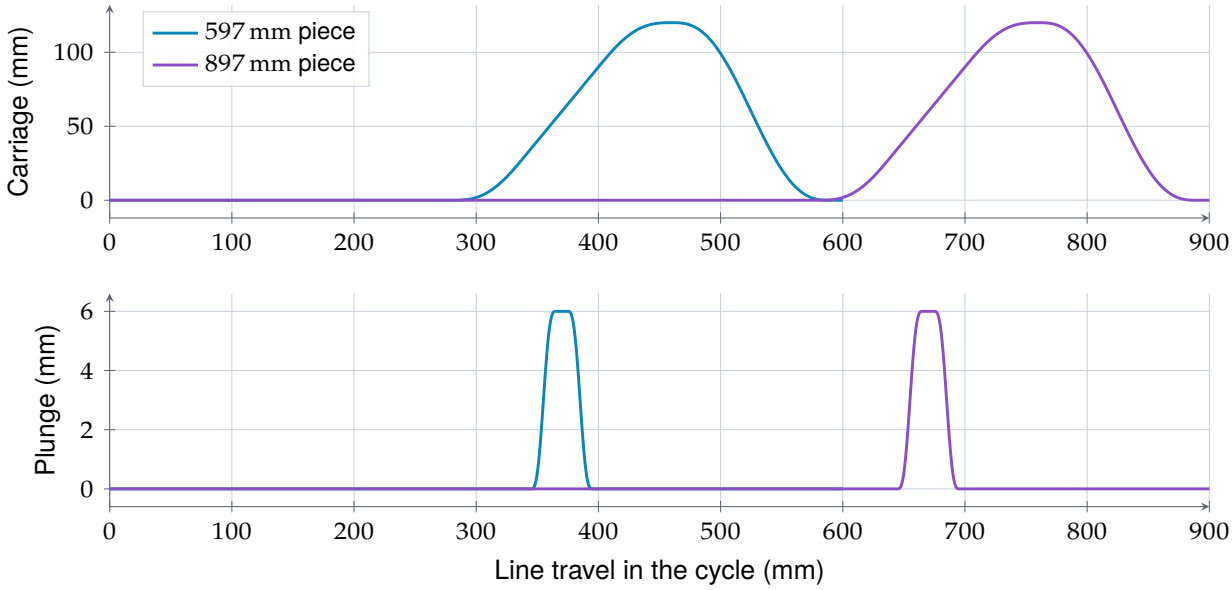


Figure 8.1. The carriage and the saw plunge for two piece lengths. The longer piece adds line travel to the wait only.

8.5 Changing the Length

A new length takes effect at the next wrap of the cycle. Thus a cycle never changes in the middle. When the crank is off, the new length takes effect immediately. The firmware also adjusts the base of the crank count, so a driven crank motor does not move when the cycle changes.

8.6 The Gap to the Cut Line

During production, the Workbench shows the distance from the blade to the cut line. With $o = C - D$, the line position at the start of the sync is $o + S_0$, where S_0 is the start of the sync in counts. The gap is the line travel to that point, less the travel the carriage still needs to reach its sync position:

$$g = (N - \varphi) - (T_c(S_0) - T_c(x))\,\frac{q}{c_c}, \tag{8.3}$$

with N the line position of the next sync start, T_c the carriage table and c_c the counts per millimetre of the carriage. During the sync, the gap is zero, and the display shows the line travel left in

the sync.

> **NOTE:** The speed limit in cut-length mode is a line speed: $v_{\max} = N_{\max} D_{\text{mm}}/1000$ metres per minute, with $N_{\max}$ in cuts per minute at the design length. It does not depend on the length of the piece, because the motion after the wait always runs at the line speed as drawn.

This page is intentionally left blank.

PART II

Theory of Operation on the ClearCore

This page is intentionally left blank.

CHAPTER 9

The System

Part I gave the mathematics of a profile. Part II shows how the Teknic ClearCore runs the profile and moves Teknic ClearPath servos in exact step with one master.

9.1 The Parts

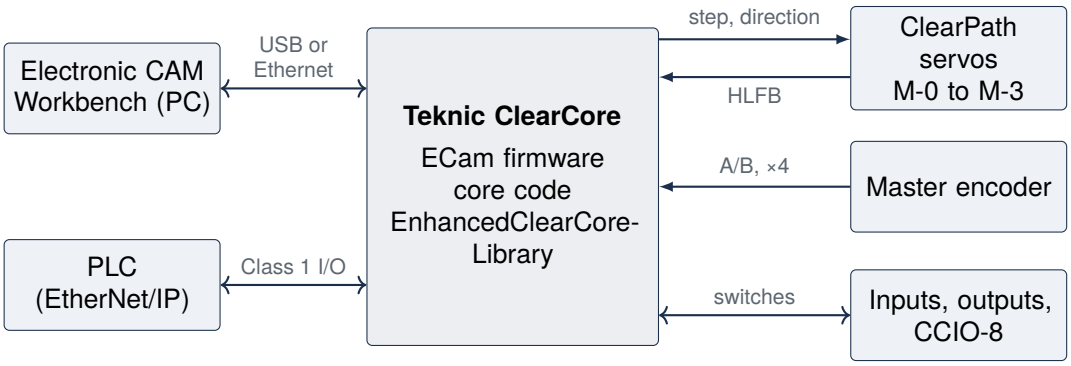


Figure 9.1. The parts of the system.

- **The Workbench** is the PC application. You design the profile and check it there. It sends the profile to the ClearCore as text, over USB or Ethernet.
- **The ClearCore** runs the ECam firmware. It keeps the profile in flash memory and runs it without the PC.
- **The ClearPath servos** receive step and direction signals from the motor connectors M-0 to M-3. Each servo closes its own position loop. Its HLFB output tells the ClearCore that the servo is enabled and has no fault.
- **The master** is an encoder on the machine, a motor that the ClearCore turns (a driven crank), or the clock of the ClearCore (a timeline).
- **A PLC** can control the machine through EtherNet/IP (chapter 15).

9.2 One Code for the PC and the ClearCore

The directory `core/` holds the laws, the profile, the analysis, the text format of a profile and the run-time engine. The Workbench and the firmware compile the same files. Thus:

- the Workbench and the ClearCore evaluate the same curve;
- the simulation in the Workbench runs the run-time engine of the firmware;
- the hash of a profile is the same on the PC and on the ClearCore.

9.3 The EnhancedClearCoreLibrary

The stock ClearCore library moves each motor with its own step generator. Each generator plans its own move. This is correct for point-to-point moves. It cannot keep several motors in step with an external master. The EnhancedClearCoreLibrary adds these functions for camming:

- `SysManager::SampleCallbackSet`: one function of the application that runs in the 5 kHz interrupt. It runs after the inputs refresh and before each connector refreshes.
- `MotorDriver::ExternalStepMode` and `SetExternalSteps`: the application gives the exact number of steps for each sample. The step generator of the library is not used. If the application sets no steps, the motor receives no steps.
- **A carry:** a sample can send at most 100 steps. The library keeps the remainder and sends it in the next samples. The stock library discarded it.
- **A direction rule:** a sample with zero steps does not change the direction output. A reversal after a sample with steps waits one sample, with its steps carried. Thus the direction never changes while steps are in flight. The stock library treated zero steps as a positive direction, and the motors drifted each cycle.
- **An enable fix:** a disable during the enable delay now ends the delay.

The application calculates one master position in each sample. It calculates the target of every axis from that position and gives the steps to every motor in the same interrupt. The library sends all the steps in the next 200 µs. Thus all axes follow one master, sample by sample, without drift.

CHAPTER 10

The Sample Loop

10.1 The Rate

The ClearCore runs its motion loop at $f_s = 5{,}000\,\text{Hz}$: one sample each 200 µs. A timer interrupt starts each sample. All the motion of the ECam firmware occurs in this interrupt.

10.2 The Order in a Sample

Figure 10.1 shows one sample. The order is important.

1. The library refreshes the CCIO-8 inputs, the analog inputs and the status.
2. The ECam callback `OnSample` runs. It reads the encoder and the inputs, filters the limit and home switches, calculates the new master position and the steps of each axis, and gives the steps to the motors.
3. Each connector refreshes. For a motor, the library loads the steps of this sample into the step timer. For an input, the library updates the filtered state.
4. The encoder position updates.

Thus the callback sees the encoder position and the filtered inputs of the previous sample. The master lags by one sample, 200 µs. The lag is constant, so it does not change the shape of the motion.

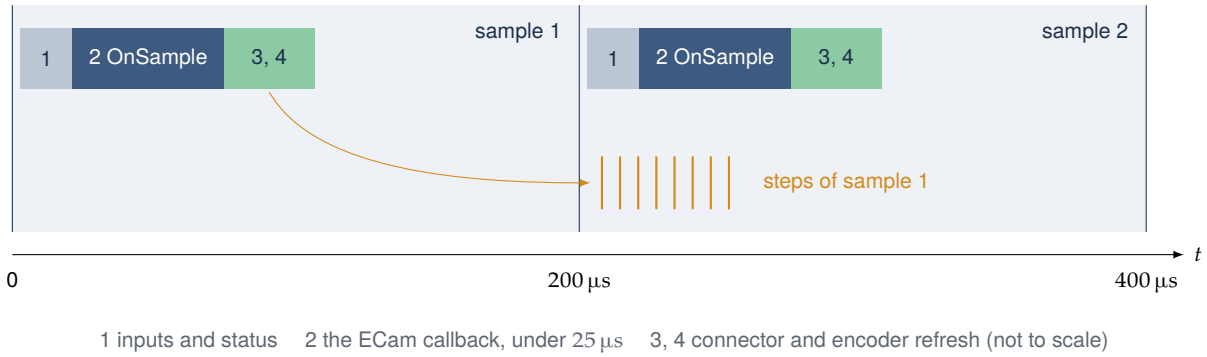


Figure 10.1. One sample. The callback calculates the steps. The library loads them, and they go out in the next period.

The callback takes less than 25 µs with three axes. Thus it uses about 12 % of the sample. The `diag` command shows the longest callback time, the number of late samples and the number of samples with carried steps.

10.3 The Main Loop

The main loop runs outside the interrupt. It does the work that can wait some milliseconds:

- the text consoles on USB and Ethernet;

- the EtherNet/IP connection;
- the emergency stop input, the motor faults and the latches of the limit, home and wait faults;
- the build of a new profile and the write to flash memory.

CAUTION

A write to flash memory stops the interrupts. During that time, the step timer repeats the pulses of the last sample, and the ClearCore does not count them. Thus the firmware writes to flash memory only when no motor moves. A commit of a profile disables the motors first.

10.4 Fixed-Point Numbers

The interrupt uses whole numbers only. A fraction is kept as a fixed-point number: Q24 is a value times 2^{24}, and Q16 is a value times 2^{16}. Table 10.1 gives the values.

Table 10.1. The fixed-point values in the sample loop.

Value	Format	Meaning
Crank phase	Q24, 64 bits	Master units in the current revolution
Crank velocity	Q24	Master units per sample
Timeline time	Q16	Samples since the start of the cycle
Timeline rate	Q16	Samples of the drawing per sample; 2^{16} is the speed as drawn
Table fraction	Q16	The position between two table entries
Axis position	32 bits	Whole motor counts

Chapter 11

The Masters

11.1 Master Units

The firmware counts the crank in *master units*. With p the encoder counts (or crank motor steps) for each revolution and a gear n/d between the encoder and the crank:

$$U = p\,n \quad \text{units for each revolution,} \qquad 1 \text{ count} = d \text{ units.} \tag{11.1}$$

Thus a gear that is not a whole number stays exact: the firmware never divides a count. U is at most 2×10^9. In cut-length mode, $U = D$, the counts of the design length, and $d = 1$.

The phase Φ is the position in the current revolution, in Q24 master units. The firmware also counts the revolutions r. Each sample, after the master moves:

$$\text{while } \Phi \geq U \cdot 2^{24}: \quad \Phi \leftarrow \Phi - U \cdot 2^{24}, \quad r \leftarrow r + 1, \tag{11.2}$$

$$\text{while } \Phi < 0: \quad \Phi \leftarrow \Phi + U \cdot 2^{24}, \quad r \leftarrow r - 1. \tag{11.3}$$

The crank angle is $\theta = 360\,\lfloor \Phi / 2^{24} \rfloor / U$.

11.2 The Encoder Master

The encoder input of the ClearCore decodes the A and B signals in hardware and counts each edge (×4). Each sample, the firmware reads the count E and adds the change to the phase:

$$\Delta\Phi = (E - E_{\text{last}}) \cdot d \cdot 2^{24}. \tag{11.4}$$

The difference is calculated in 32-bit arithmetic, so the wrap of the counter has no effect. At the start, E_{last} is set to the current count, so the axes do not jump when they engage.

The machine sets the speed of an encoder master. The firmware cannot ramp it. When you stop the cycle, the axes on an encoder master disengage at once.

11.3 The Driven Crank

With a driven crank, the ClearCore turns a crank motor at a set speed and ramp. For N rpm and a ramp of N_r rpm per second, the target velocity and the ramp in Q24 units per sample are

$$v_t = \text{round}\left(\frac{N\,U}{60 \cdot 5000}\, 2^{24} \right), \qquad a_r = \max\left(1,\ \text{round}\left(\frac{N_r\,U}{60 \cdot 5000^2}\, 2^{24} \right) \right). \tag{11.5}$$

Each sample, the velocity moves toward the target by at most a_r, and the phase adds the velocity:

$$v \leftarrow v + \text{clamp}(v_t - v,\ -a_r,\ a_r), \qquad \Phi \leftarrow \Phi + v. \tag{11.6}$$

Figure 11.1 shows a start, a run and a stop of the carton pusher crank: 32 rpm with a ramp of 20 rpm/s.

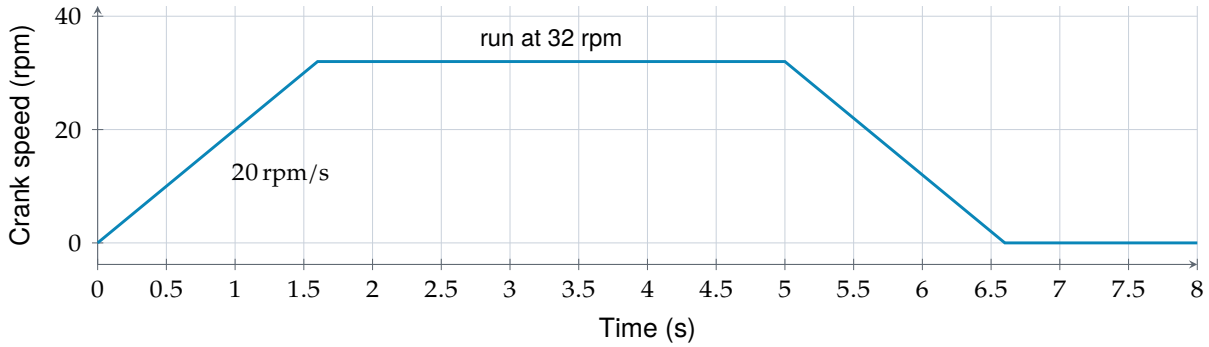


Figure 11.1. The speed of a driven crank. The acceleration term $y'\alpha$ of equation (2.8) applies on the ramps.

The steps of the crank motor come from the whole units only:

$$n_{\text{crank}} = \left\lfloor \frac{B + r\,U + \lfloor \Phi / 2^{24} \rfloor}{d} \right\rfloor, \tag{11.7}$$

with B a base count that keeps the motor continuous when the cycle of cut-length mode changes. The step output of each sample is the change of n_{crank}. The fraction of a unit in Φ goes to the axes, so they follow the crank motor more smoothly than one unit.

11.4 The Timeline

A timeline counts time in samples, in Q16. Each sample:

$$t \leftarrow t + R, \qquad R = k \cdot 2^{16}, \tag{11.8}$$

with k the speed override, from 0 to 10. When you change the override, the rate moves to the new value by $2^{16}/1250$ each sample, so it reaches full speed in about 0.25 s.

11.4.1 The Trigger

A timeline is *armed* after the start. A trigger starts the cycle: a command, or an edge on an input. At the trigger, the axes engage and the rate goes to the target at once, because every axis starts from rest.

11.4.2 The End of the Cycle

- A repeating timeline subtracts the cycle and continues. It counts the cycles, so an open axis keeps its advance.
- A one-shot timeline stops at the end of the cycle and is *done*. The next trigger starts a new cycle.

11.4.3 Waits

A wait stops the timeline at a time until a condition is true: an input, a bit from the PLC, or a CCIO-8 input. In each sample, the clock moves from t_0 to t_1. If a wait is in that step and its condition is false, the clock stops exactly at the time of the wait. When the condition becomes true, the timeline continues at its rate. A wait whose condition is already true does not stop the

clock. A wait can have a timeout. At the timeout, the timeline stops and the firmware latches a wait fault.

Figure 11.2 shows the example `pick_and_place`: three axes on a one-shot timeline of 2.4 s, started by an input.

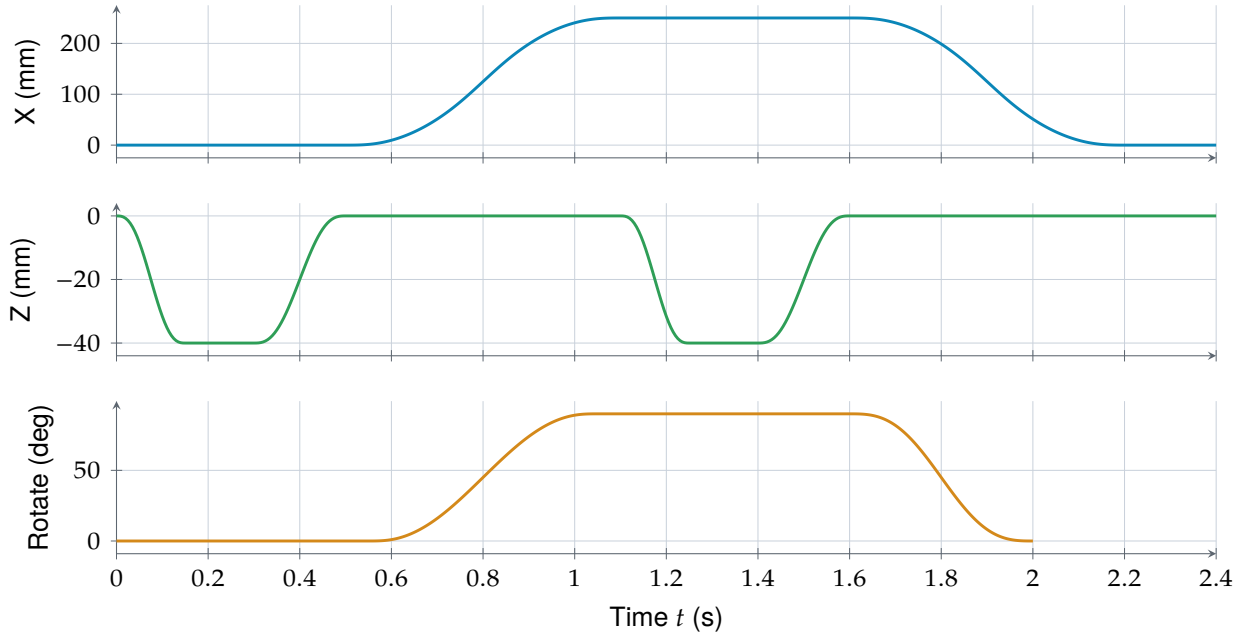


Figure 11.2. The three axes of a pick-and-place cycle on one timeline.

11.5 Simulation

In simulation the master runs and the positions change, but the motors receive no steps and the outputs stay off. A simulated encoder can turn the master without a machine.

This page is intentionally left blank.

Chapter 12

Followers and Step Generation

12.1 The Position Tables

The interrupt has 200 µs for all axes. It does not evaluate the laws. When the ClearCore receives a profile, it builds a table of $K + 1 = 2049$ whole motor counts for each axis, in double precision, outside the interrupt:

$$T[k] = \text{round}\left(c\, y\left(\frac{X\, k}{K} \right) \right), \qquad K = 2048. \tag{12.1}$$

The last entry comes from the end of the last segment. The advance of each cycle, in counts, is $T[K] - T[0]$.

12.2 Interpolation

At a master position x, the firmware finds the table interval and the fraction f in it, in Q16:

$$i = \left\lfloor \frac{x\, K}{X} \right\rfloor, \qquad f = \frac{x\, K}{X} - i, \qquad y_{\text{counts}} = T[i] + \lfloor (T[i+1] - T[i])\, f \rfloor + r\, (T[K] - T[0]), \tag{12.2}$$

with r the revolution (or the cycle of a timeline). On a crank, x is the phase in whole master units, and on a driven crank the fraction of a unit is included in f. On a timeline, x is the time.

The error of linear interpolation between two points of a smooth curve is at most $|y''|\,\Delta^2/8$, with Δ the interval of the table. For the carton pusher, $\Delta = 360/2048 = 0.176°$ and the peak acceleration is $0.0902 \times 80 = 7.21$ counts/deg^2. Thus the error is at most

$$7.21 \times 0.176^2/8 \approx 0.03 \text{ counts}. \tag{12.3}$$

The rounding of the table entries adds at most half a count. Thus the table follows the curve to less than one count.

12.3 Steps Are Differences

In each sample, the firmware calculates the target count of each engaged axis. The steps of the sample are the change of the target:

$$n_{\text{steps}} = y_{\text{counts}}(\text{now}) - y_{\text{counts}}(\text{last sample}). \tag{12.4}$$

The sum of the steps is always the change of the target. Thus an axis cannot gain or lose a count through rounding. When an axis engages, the firmware sets the last target to the current target, so the axis does not jump.

Figure 12.1 shows the steps of each sample for the carton pusher at 32 rpm: at most 8 steps in a sample, with $c = 80$ counts/mm.

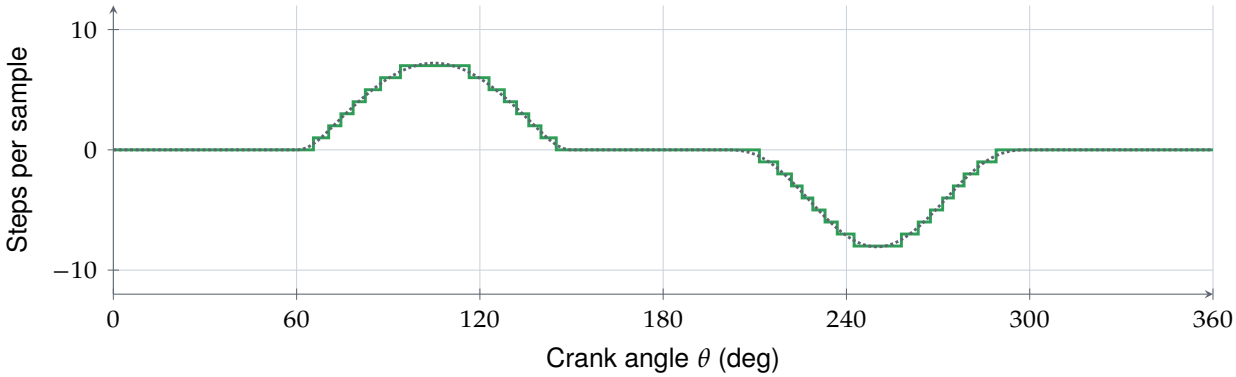


Figure 12.1. Steps per sample of the carton pusher at 32 rpm (dotted: the exact rate). The limit of the ClearCore is 100 steps per sample.

12.4 The Electronic Gear at Run Time

A gear axis does not use a table. It uses the exact whole-number method of section 4.7. The master units it receives are the whole units of the master movement in this sample. The fraction stays for the next sample. The encoder, the driven crank and the hand jog all feed this movement, so a ramp or a jog needs nothing more.

12.5 Step Output in the Library

After the callback, each motor connector refreshes. In external step mode, the library:

1. adds the steps carried from earlier samples;
2. applies the direction rule: zero steps do not change the direction, and a reversal right after a sample with steps waits one sample, with its steps carried;
3. limits the count to the maximum of a sample and carries the remainder;
4. writes the count to the step timer. The pulses go out during the next 200 µs.

The ECam firmware uses the normal clock rate of the motor connectors: 500 kHz. Thus a sample can send at most 100 steps, and the step rate is at most 500,000 steps per second. The library also updates the commanded position of the motor from the steps that it sends.

> **NOTE:** A carry is not an error. It delays some steps by one or more samples. The axis then catches up exactly. The `diag` command counts the samples with a carry.

CHAPTER 13

Running a Machine

This chapter gives the states of the master, the start and stop of a cycle, the cam position, the jogs, homing and the protection functions.

13.1 The States

The crank and the timeline each have a state. Figure 13.1 shows them.

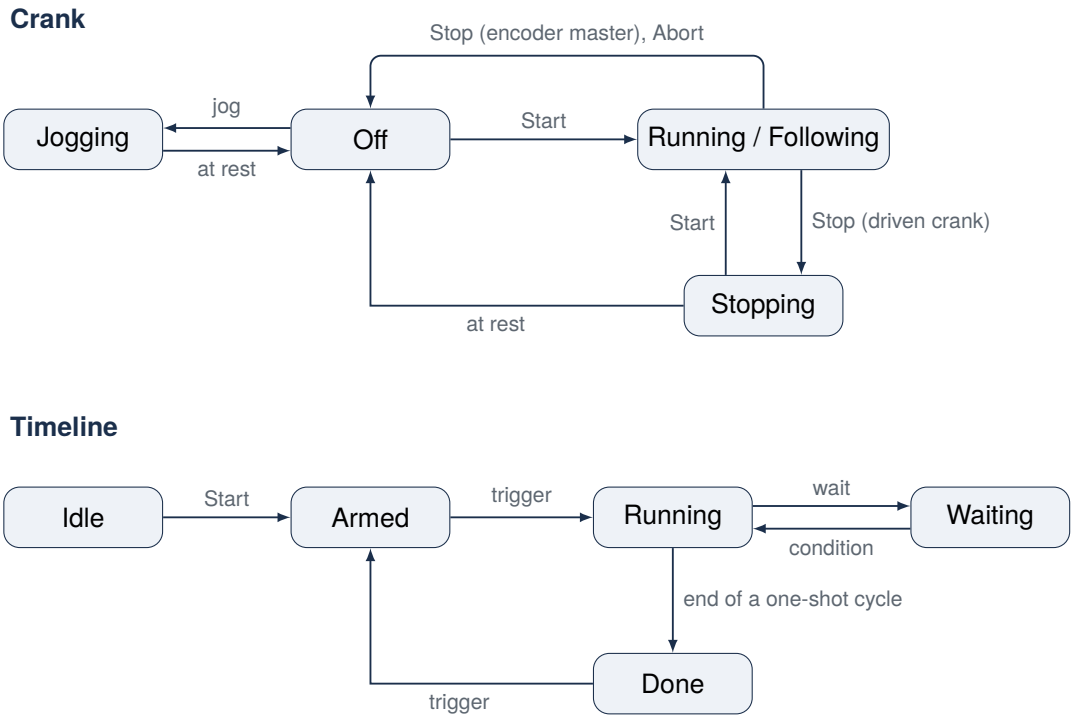


Figure 13.1. The states of the crank and the timeline.

13.2 Start

Start does these steps:

- On a crank that is off, it latches the encoder, engages the axes on the crank and starts the master.
- On a timeline that is idle or done, it arms the timeline. The axes engage at the trigger.

The firmware refuses the start when: no profile is loaded, a fault is latched, the motors are not enabled (except in simulation), an axis jog moves, an axis is on a limit switch, an axis that must home is not homed, or an axis is not at its cam position.

13.3 The Cam Position

An axis is at its cam position when its position is within one count of the position that the profile gives at the current master position. The firmware checks an axis only if a jog, a zero or a homing has moved it, or if it is a timeline axis while the timeline is not running. If an axis is not at its cam position, the Workbench offers to move all axes to their cam positions.

13.4 Stop and Abort

- **Stop** is a cycle stop. A running timeline completes its cycle and then rests at the end. A driven crank ramps down at its ramp. The axes on an encoder master disengage at once, because the firmware cannot ramp the machine.
- **A second stop**, or a stop with no cycle to complete, is an immediate stop. The timeline rate ramps to zero in the stop time (500 ms by default).
- **Abort** stops everything at once, without a ramp.

13.5 Enable and Disable

Enable clears the fault, clears the alerts of the ClearPath motors and enables each motor that the profile uses. Then it waits up to 1 s until the HLFB of each motor shows that the motor is ready. If a motor does not become ready, the firmware disables all motors and latches a fault.

Disable aborts the motion and disables all motors. It also clears the homed state of each axis.

13.6 Jogs

- **The hand jog** moves the master (the crank or the timeline), and all engaged axes follow it. The jog uses a speed profile that stops exactly on the target: $v \leftarrow \text{step}(v,\ \text{sign}(e)\ \min(v_{\max}, \sqrt{2a|e|}),\ a)$, with e the distance to the target.
- **The axis jog** moves one motor alone, when nothing is engaged. It uses the velocity and acceleration limits of the axis, never more than 53,333 counts per second. A move to a position lands on the exact count. The jog stops if the PC does not send a keep-alive within 500 ms.

13.7 Homing

Homing uses the axis-jog generator. It has these phases:

1. If the axis starts on the switch, it moves off the switch.
2. It searches for the switch at the fast speed.
3. It stops, and moves back until the switch releases, plus a back-off distance.
4. It moves onto the switch at the slow speed and latches the position.
5. It sets the position from the latch, without steps. Then it can move to the start of the cam.

Axes with the same order number home together. A failure (a limit switch, a switch not found, or a switch that does not release) latches a home fault and stops the remaining axes.

13.8 Protection

Table 13.1. The protection functions

Event	Action	Motors
Emergency stop	Abort at once. The main loop reads the input.	Disabled
Motor fault	An enabled motor with its HLFB off. Abort.	Disabled
Limit switch	In the interrupt, filtered over 3 samples. The master stops in the limit stop time; an encoder master's axes ramp each one down.	Enabled
Wait timeout	The timeline stops.	Enabled
Home failure	The homing stops.	Enabled
Lost PLC connection	An immediate stop, then disable at rest.	Disabled

CAUTION

A board input passes through the filter of the library and the filter of the firmware. Thus a limit switch acts about 1.2 ms to 1.4 ms after its edge. Include this time, at the highest speed, in the distance from the switch to the hard stop.

This page is intentionally left blank.

Chapter 14

Profile Storage and Integrity

14.1 The Text Format

The Workbench sends a profile to the ClearCore as lines of text. Each line starts with a keyword: `master`, `timeline`, `axis`, `seg`, `out`, `wait`, `input`, `home`, `limits` and others. Numbers use 17 significant digits, so a value is the same on the PC and on the ClearCore. These limits apply:

- a line has at most 159 characters;
- a profile has at most 32 KB of text;
- an axis has at most 64 segments.

14.2 The Hash

The hash of a profile is the 32-bit FNV-1a hash of each line, each followed by a line feed:

$$h_0 = 2166136261, \qquad h \leftarrow (h \oplus b) \times 16777619 \bmod 2^{32} \tag{14.1}$$

for each byte b. The Workbench and the ClearCore calculate the same hash. Thus the Workbench can show whether the ClearCore runs the profile on the screen.

14.3 The Commit

A commit makes the received profile the active profile:

1. The firmware checks the profile and makes it valid with the same code as the Workbench.
2. It refuses the commit while a motor moves.
3. It disables the motors, and stops the run-time engine in the interrupt.
4. It builds the tables (chapter 12).
5. It sets the limits, the waits, the outputs, the trigger and the direction of the encoder. It puts each motor in external step mode.
6. It writes the profile to flash memory.

14.4 The Flash Store

The profile is kept in 32 KB of flash memory at address `0x78000`. A header of 16 bytes holds a marker, a version, the length and a checksum of the text. The checksum is the FNV-1a hash of the text, so it is equal to the hash of the profile. After the write, the firmware reads the memory back and compares it.

At power-up, the firmware reads the store before Ethernet starts. It loads the profile through the same path as a commit. The motors stay disabled, and no motion starts.

This page is intentionally left blank.

CHAPTER 15

Control from a PLC

The ClearCore is an EtherNet/IP adapter. A PLC opens a Class 1 connection and exchanges two assemblies with it each requested packet interval. A Rockwell Micro800 can use the connection with SINT or DINT data.

15.1 The Assemblies

Table 15.1. The assemblies of the ClearCore

Instance	Direction	Size	Content
150	PLC to ClearCore	64 bytes	Command: control bits, setpoints, wait bits
100	ClearCore to PLC	160 bytes	Status: states, positions, faults
151	Configuration	0 bytes	–

The firmware updates the status each millisecond.

15.2 Who Has Control

The PLC has control when the connection is open, the PLC is in Run and the bit PLC_CONTROL is on. While the PLC has control, the consoles refuse the commands that start or change motion.

Table 15.2. The control bits of the command assembly

Bit	Name	Action
0	ENABLE	Level. On: enable. Off: always disable.
1	PLC_CONTROL	Level.
2	START	Rising edge.
3	STOP	Rising edge. Always acts.
4	ABORT	Rising edge. Always acts.
5	CLEAR_FAULT	Rising edge.
6	SIMULATE	Both edges.
7	JOG	Both edges. The falling edge always acts.
8	TRIGGER	Rising edge.
9	HOME	Rising edge.

In one packet, the firmware does the actions in this order: clear the fault, simulation, enable, setpoints, start, trigger, home, jog. Bits that are already on in the first packet after the connection

opens are not edges. A setpoint applies when it changes. A setpoint of zero gives the value of the profile. The wait bits do not need control.

15.3 Loss of the Connection

If the connection closes, times out or the PLC goes to Program while the PLC has control:

- If the machine moves, the firmware makes an immediate stop and latches a fault. It disables the motors when they are at rest.
- If the machine is at rest but enabled, the firmware disables the motors.

15.4 A Typical Sequence

1. Set PLC_CONTROL.
2. Set ENABLE. Wait until the status shows that the motors are enabled.
3. If the axes must home, set HOME. Wait until the status shows that they are homed.
4. Set START. For a timeline, set TRIGGER for each cycle, or use the trigger input.
5. Write the speed, the override or the cut length as setpoints during production.
6. Set STOP to stop at the end of the cycle.

Appendix A

Coefficients of the Laws

The values below come from the code (`tools/docs_data`, file `law_peaks.dat`). For a segment with rise h and span β:

$$y'_{\max} = C_v \frac{h}{\beta}, \qquad y''_{\max} = C_a \frac{h}{\beta^2}, \qquad y'''_{\max} = C_j \frac{h}{\beta^3}.$$

Table A.1. The peak coefficients of the laws

Law	C_v	C_a	C_j
Linear	1.0000	0.0000	0.000
Simple harmonic	1.5708	4.9348	15.503
Cycloidal	2.0000	6.2832	39.478
Modified trapezoid	2.0000	4.8881	61.426
Modified sine	1.7596	5.5280	69.466
Polynomial 3-4-5	1.8750	5.7735	60.000
Polynomial 4-5-6-7	2.1875	7.5132	52.500
Blend (at rest)	1.8750	5.7735	60.000
Spline (one segment)	1.5000	6.0000	12.000

This page is intentionally left blank.

Glossary

Advance
The change of position of an open profile in one cycle. An index feed uses it.

Axis
One servo motor and its mechanics. It follows the master. Also called a follower.

Blend
A motion law that starts and ends at the velocities of its neighbours. It joins a dwell to a linear segment, for example.

Cam position
The position that the profile gives for an axis at the current master position. An axis must be at its cam position before a start.

Carry
Steps that the library keeps for the next sample, because a sample can send at most 100 steps or because the direction changes.

Class 1 connection
An EtherNet/IP I/O connection that sends assembly data at a fixed interval. A PLC uses it to control the ClearCore.

Closed profile
A profile that ends at its start position.

Count
One step of a motor. Motor positions are in counts.

Crank
A master that turns once for each cycle: an encoder on the machine, or a motor that the ClearCore turns (a driven crank).

Cubic spline
A curve of cubic polynomials through a list of knots, with continuous position, velocity, and acceleration.

Cut-length mode
A mode of a crank master for a flying saw. The cycle stretches to the length of the piece.

Cyclic axis
An axis whose profile repeats: each axis on a crank, and an axis on a repeating timeline that spans the full cycle.

Dwell
A segment in which the axis does not move.

Electronic gear
An axis that follows the crank at a fixed ratio, calculated in whole numbers.

Engage
To start to follow the master. An axis engages without a jump.

HLFB
High-Level Feedback. An output of a ClearPath servo that shows that the servo is enabled and has no fault.

Jerk
The rate of change of the acceleration.

Knot
The end of a segment. Knot 0 is the start of the profile.

Law
The shape of the motion in a segment, defined on the unit square.

Master
The source of the progress of the machine cycle: a crank or a timeline.

Master unit
The unit in which the firmware counts the crank: one count of the encoder or crank motor divided by the denominator of the crank gear.

Open profile
A profile that ends at a position different from its start. It advances each cycle.

Peak coefficient
The largest value of the velocity (C_v), acceleration (C_a), or jerk (C_j) of a law on the unit square.

Pitch curve
The path of the center of the roller of a plate cam.

PLC
Programmable logic controller.

Pressure angle
The angle between the direction of motion of the follower and the normal to the cam at the contact point.

Profile
The position of an axis as a function of the master position.

Q16, Q24
Fixed-point numbers: a value times 2^{16} or 2^{24}, kept as a whole number.

Sample
One period of the 5 kHz motion loop: 200 µs.

Segment
A part of a profile between two knots, with one law.

Simulation
A mode in which the master runs and the positions change, but the motors receive no steps.

Sync
The part of a flying-saw cycle in which the carriage moves at the speed of the material.

Timeline
A master that is a clock. A trigger starts its cycle.

Undercut
A part of a plate cam that the roller cannot follow, because the radius of curvature of the pitch curve is smaller than the radius of the roller.

Wait
A point on a timeline where the clock stops until a condition is true.

This page is intentionally left blank.

Bibliography

ASD (AeroSpace, Security and Defence Industries Association of Europe). *ASD-STE100: Simplified Technical English.* Issue 8. Brussels: ASD, 2021.

Burden, Richard L., and J. Douglas Faires. *Numerical Analysis.* 9th ed. Boston: Brooks/Cole, 2011.

The Chicago Manual of Style. 18th ed. Chicago: University of Chicago Press, 2024.

Norton, Robert L. *Cam Design and Manufacturing Handbook.* 2nd ed. New York: Industrial Press, 2009.

ODVA. *The CIP Networks Library.* Vol. 2, *EtherNet/IP Adaptation of CIP.* Ann Arbor, MI: ODVA.

Rothbart, Harold A., ed. *Cam Design Handbook.* New York: McGraw-Hill, 2004.

Sherman, Jack, and Winifred J. Morrison. "Adjustment of an Inverse Matrix Corresponding to a Change in One Element of a Given Matrix." *Annals of Mathematical Statistics* 21, no. 1 (1950): 124–27.

Sweeney, Adam G. *QuickEIP Programming and Implementation Manual.* Document QE-PIM-200, revision A. 2026.

———. *Electronic CAM Workbench.* Version 0.11.1. Source code repository, MIT License, 2026.

———. *EnhancedClearCoreLibrary.* Source code repository, 2026.

Teknic, Inc. *ClearCore User Manual.* Victor, NY: Teknic.

Teknic, Inc. *ClearPath-SD User Manual.* Victor, NY: Teknic.